Code Reference: docs/build_complete_guide.py

A source-level explanation with function details, file communication, variables, endpoints, and debugging notes.

Item	Explanation
File	docs/build_complete_guide.py
Purpose	Tracked source/configuration file in the OMB Portfolio Builder repository.
Lines	2,412
Size	163,977 bytes
Talks to	builder frontend, local backend, public website runtime, portfolio catalog, Cloudflare/AI layer, GitHub/release layer
Functions	75
Variables/constants	0
API mentions	6
Signals	434

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Imports, requires, and external dependencies

Item	Explanation
Line 3	import html
Line 4	import json
Line 5	import math
Line 6	import re
Line 7	import shutil
Line 8	import subprocess
Line 9	import textwrap

API endpoints mentioned or called

Item	Explanation
/api/...	Line 93. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/code/save	Line 449. Saves source code into the local compile workspace.
/api/code/compile	Line 642. Compiles or runs a source file and returns terminal output.
/api/code/compile	Line 654. Compiles or runs a source file and returns terminal output.
/api/portfolio-ai	Line 666. Handles AI assistant questions.
/api/portfolio-ai	Line 697. Handles AI assistant questions.

Important implementation signals

Item	Explanation
Rich text at line 12	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: from PIL import Image, ImageDraw, ImageFont
Rich text at line 18	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: from docx.shared import Inches, Pt, RGBColor

Item	Explanation
Rich text at line 21	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: from reportlab.lib import colors
Installer at line 44	Windows setup, update, shortcut, or installation behavior. Evidence: ".github/workflows/build-windows-builder.yml": "GitHub Actions workflow that builds the Windows installer and portable application, checks the stable download link, enforces release version bumps, uploads artifacts, and
Git command at line 46	Repository state, publishing, branch checks, or release automation. Evidence: ".gitignore": "Git ignore rules for build outputs, temporary files, and local-only data that should not be committed.",
Cloudflare or AI at line 49	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "BRANCHING.md": "Human-readable branch model explaining development, main, feature branches, and release flow.",
Git command at line 50	Repository state, publishing, branch checks, or release automation. Evidence: "Backgrounds/.gitkeep": "Keeps the Backgrounds folder in Git even when no background image files are present.",
Cloudflare or AI at line 51	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "README.md": "Main README for the standalone builder application: install, updates, workspaces, text editing, publishing, build commands, branch model, and uninstall.",
Cloudflare or AI at line 52	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "_headers": "Cloudflare Pages style HTTP headers for security and caching behavior on the public website.",
Installer at line 61	Windows setup, update, shortcut, or installation behavior. Evidence: "assets/omb-app-icon.ico": "Windows ICO application icon used by Electron Builder and shortcuts.",
Installer at line 63	Windows setup, update, shortcut, or installation behavior. Evidence: "build/installer.nsh": "NSIS installer customization script for update behavior, shortcut placement, old install detection, and setup details.",
Cloudflare or AI at line 65	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "cloudflare/README.md": "Cloudflare setup notes for deploying the AI Worker endpoint and wiring the public site to the backend.",
Cloudflare or AI at line 66	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "cloudflare/portfolio-ai-worker.js": "Cloudflare Worker backend for the public portfolio AI assistant and fallback intelligence path.",
Cloudflare or AI at line 67	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "cloudflare/wrangler.toml": "Wrangler configuration used to deploy the Cloudflare Worker.",
Git command at line 68	Repository state, publishing, branch checks, or release automation. Evidence: "docs/.gitkeep": "Keeps the docs folder in Git even before generated documents are added.",
Installer at line 71	Windows setup, update, shortcut, or installation behavior. Evidence: "installer-notes.txt": "Installer license/notes text shown during Windows setup.",
Compiler or simulator at line 73	Part of the Compile Code workspace or language/tool detection. Evidence: "package.json": "Node/Electron package manifest with scripts, dependencies, version, installer settings, and extra resources.",
Security or auth at line 81	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "server.mjs": "Local backend server used by the builder for drafts, parsing, publishing, compiler execution, authentication checks, and file operations.",
Git command at line 90	Repository state, publishing, branch checks, or release automation. Evidence: ("High-Level System Map", "The system has two personalities: a private builder app and a public website. The builder is where content is created and verified. The website is what recruiters see after publishing.", "Owner
Security or auth at line 92	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("Private Builder Versus Public Website", "The builder includes editing controls, local draft storage, authentication checks, compiler tools, and publishing actions. The public website removes those controls and only dis
Git command at line 93	Repository state, publishing, branch checks, or release automation. Evidence: ("Local Backend Role", "The frontend cannot safely do everything by itself. The local backend handles file writes, repository operations, compiler execution, publishing checks, and generated site output.", "template-prev
Git command at line 95	Repository state, publishing, branch checks, or release automation. Evidence: ("GitHub Pages Publishing", "The target repository receives static files. GitHub Pages then serves those files over HTTPS. The builder does not run on the public site.", "Local publish mirror -> git commit -> git push ->
Cloudflare or AI at line 96	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("Cloudflare Role", "Cloudflare can sit in front of the domain and can also host Worker endpoints such as the portfolio AI backend. DNS points the custom domain to the website host.", "Visitor DNS lookup -> Cloudflare DN
Cloudflare or AI at line 97	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("AI Backend Role", "The website AI should answer from portfolio context when the question is about Maurice's work and use general knowledge when the question is general. The Worker endpoint keeps API keys off the public
Installer at line 99	Windows setup, update, shortcut, or installation behavior. Evidence: ("Installer And Update Role", "The installer places the app in the per-user AppData Programs folder. Updates should replace that install instead of creating duplicate copies.", "GitHub Release -> installer download -> up

Item	Explanation
Cloudflare or AI at line 100	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("Branch Model", "Development collects work. Main is the release branch. Feature branches should merge into development first. Main should receive only development.", "feature branch -> development -> main -> release wor
Security or auth at line 101	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("Security Boundary", "Editing is local and authenticated. Public visitors should not get editing tools. Publishing requires GitHub write access to the target repository.", "Visitor can read public site\nOwner can edit b
Cloudflare or AI at line 109	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Cloudflare Role": "cloudflare-ai-flow",
Cloudflare or AI at line 110	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "AI Backend Role": "cloudflare-ai-flow",
Installer at line 112	Windows setup, update, shortcut, or installation behavior. Evidence: "Installer And Update Role": "release-pipeline",
Cloudflare or AI at line 113	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Branch Model": "release-pipeline",
Git command at line 118	Repository state, publishing, branch checks, or release automation. Evidence: ("git status", "Shows what branch you are on and whether files are changed, staged, or clean.", "Run it before edits, before commits, and before merges."),
Git command at line 119	Repository state, publishing, branch checks, or release automation. Evidence: ("git switch development", "Moves your working copy to the development branch.", "Use development as the integration branch before anything goes to main."),
Git command at line 120	Repository state, publishing, branch checks, or release automation. Evidence: ("git switch -c codex/example-change", "Creates a new feature branch from the branch you are currently on.", "Use this before changing the builder so development stays stable."),
Git command at line 121	Repository state, publishing, branch checks, or release automation. Evidence: ("git add <file>", "Stages a changed file for the next commit.", "Only stage files that belong to the current change."),
Git command at line 122	Repository state, publishing, branch checks, or release automation. Evidence: ("git commit -m \"message\"", "Records staged changes as a named checkpoint.", "Write messages that explain why the change exists."),
Git command at line 123	Repository state, publishing, branch checks, or release automation. Evidence: ("git push origin <branch>", "Uploads your branch to GitHub.", "This lets GitHub run workflows and enables pull requests."),
Git command at line 124	Repository state, publishing, branch checks, or release automation. Evidence: ("git pull --ff-only", "Updates the current branch only if Git can fast-forward safely.", "Avoids surprise merge commits."),
Git command at line 125	Repository state, publishing, branch checks, or release automation. Evidence: ("git fetch --tags --force", "Downloads branch and tag metadata from GitHub.", "Used before checking whether a release tag already exists."),
Compiler or simulator at line 126	Part of the Compile Code workspace or language/tool detection. Evidence: ("pnpm install --frozen-lockfile", "Installs Node dependencies exactly as pinned in pnpm-lock.yaml.", "Makes release builds repeatable."),
Installer at line 129	Windows setup, update, shortcut, or installation behavior. Evidence: ("pnpm run installer", "Builds the NSIS Windows installer.", "Use this when testing installation and update behavior."),
Cloudflare or AI at line 132	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("wrangler deploy", "Deploys the Cloudflare Worker backend.", "Use it when the AI endpoint code changes."),
Compiler or simulator at line 133	Part of the Compile Code workspace or language/tool detection. Evidence: ("gcc file.c -o app.exe", "Compiles C source into a Windows executable.", "The builder uses similar logic when C tools are available."),
Compiler or simulator at line 135	Part of the Compile Code workspace or language/tool detection. Evidence: ("javac Main.java", "Compiles Java source into .class bytecode.", "Java has a compile step before running."),
Compiler or simulator at line 136	Part of the Compile Code workspace or language/tool detection. Evidence: ("java Main", "Runs compiled Java bytecode.", "The builder shows this output in the terminal panel."),
Compiler or simulator at line 137	Part of the Compile Code workspace or language/tool detection. Evidence: ("python script.py", "Runs Python source directly.", "Python is interpreted by default."),
Compiler or simulator at line 138	Part of the Compile Code workspace or language/tool detection. Evidence: ("node script.js", "Runs JavaScript through Node.js.", "Useful for builder scripts and JavaScript examples."),
Compiler or simulator at line 139	Part of the Compile Code workspace or language/tool detection. Evidence: ("iverilog -g2012 design.sv -o sim.vvp", "Compiles Verilog/SystemVerilog into an Icarus simulation file.", "Prepares HDL for simulation."),
Compiler or simulator at line 140	Part of the Compile Code workspace or language/tool detection. Evidence: ("vvp sim.vvp", "Runs Icarus Verilog simulation output.", "This is how HDL terminal output appears."),
Git command at line 141	Repository state, publishing, branch checks, or release automation. Evidence: ("git push --dry-run origin main", "Tests whether GitHub would accept a push without changing the remote.", "Used during authorization checks."),
Git command at line 142	Repository state, publishing, branch checks, or release automation. Evidence: ("git remote -v", "Shows which GitHub repository the local folder talks to.", "Important for publishing targets."),

Item	Explanation
Git command at line 143	Repository state, publishing, branch checks, or release automation. Evidence: ("git branch --show-current", "Prints the active branch name.", "The builder must know whether it is pushing the right branch."),
Git command at line 144	Repository state, publishing, branch checks, or release automation. Evidence: ("git tag --list builder-v*", "Lists builder release tags.", "Release workflows use this to prevent duplicate releases."),
Installer at line 145	Windows setup, update, shortcut, or installation behavior. Evidence: ("Get-ChildItem", "PowerShell command that lists files and folders.", "Used to inspect workspaces and installer output."),
Process execution at line 147	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: ("Start-Process", "PowerShell command that starts another program.", "The updater uses detached process launching."),
Cloudflare or AI at line 152	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Profile identity and contact details", "Front-page hero text", "Fun facts block", "Portfolio areas", "Project categories", "Project creation flow", "Project overview", "Design sections", "Simulation sections", "Files an
Installer at line 157	Windows setup, update, shortcut, or installation behavior. Evidence: ("The app does not open", "Check whether the installed executable exists under AppData Local Programs. If it does, start it directly. If it fails, reinstall from the latest GitHub Release."),
Installer at line 158	Windows setup, update, shortcut, or installation behavior. Evidence: ("Update does not restart", "The update handoff must close Electron, run the installer, and then launch the new executable. Check the updater log and whether another builder process is still running."),
Security or auth at line 159	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("GitHub authentication succeeds but builder cannot push", "Authentication and repository freshness are different. Pull or load from target first if the remote branch is ahead."),
Local storage at line 161	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: ("Website changes show on desktop but not phone", "Check cache, confirm the published files changed, and verify the mobile renderer is using the same projects.json and CSS."),
Cloudflare or AI at line 162	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("AI endpoint unavailable", "Verify the Worker route, environment secrets, Cloudflare deployment, and browser network errors."),
Local storage at line 163	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: ("Compiler takes too long", "Check whether the first run is installing or detecting tools. Cached runs should be faster after the first successful compile."),
Compiler or simulator at line 164	Part of the Compile Code workspace or language/tool detection. Evidence: ("SystemVerilog output missing", "Check that Icarus Verilog and vvp are available and that the file is a simulation testbench or has a runnable top module."),
Rich text at line 165	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: ("Right-click formatting does not apply", "Confirm the active editor saved its selection before the menu changed font, size, or color."),
Rich text at line 166	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: ("Links are treated as formulas", "URL detection should win before formula detection. Pasted http, https, and www links must become normal links."),
Installer at line 168	Windows setup, update, shortcut, or installation behavior. Evidence: ("Installer reports an old installation", "The installer searches per-user and legacy locations. Remove stale install records only after confirming the active app path."),
Git command at line 176	Repository state, publishing, branch checks, or release automation. Evidence: "What HTML does", "What CSS does", "What JavaScript does", "What JSON does", "What Markdown does", "What YAML does", "What TOML does", "What an executable is", "What an installer is", "What a portable app is", "What a lo
Git command at line 182	Repository state, publishing, branch checks, or release automation. Evidence: ("Electron vs pure web builder", "Electron gives desktop file access and packaging. A pure web builder would be easier to access anywhere, but browser security would block many local file, compiler, and Git operations.")
Installer at line 184	Windows setup, update, shortcut, or installation behavior. Evidence: ("Per-user install vs Program Files", "Per-user installs update without admin prompts. Program Files feels traditional but causes UAC friction and update failures."),
Git command at line 187	Repository state, publishing, branch checks, or release automation. Evidence: ("Git authentication cache vs always prompt", "Caching reduces annoyance. Always prompting is stricter but makes normal publishing painful."),
Cloudflare or AI at line 188	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("Cloudflare Worker AI vs browser-only AI", "Worker AI hides secrets and can access server-side providers. Browser-only AI would expose keys or be limited to local browser models."),
Git command at line 203	Repository state, publishing, branch checks, or release automation. Evidence: ("git", "Git target repo\ncommit + push", 1040, 140, 250, 135, "#FEF3C7"),
Cloudflare or AI at line 206	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("ai", "Cloudflare Worker\nportfolio AI endpoint", 1040, 565, 250, 130, "#CCFBF1"),
Git command at line 212	Repository state, publishing, branch checks, or release automation. Evidence: ("backend", "git", "publishes"),
Git command at line 213	Repository state, publishing, branch checks, or release automation. Evidence: ("git", "site", "hosts files"),
Cloudflare or AI at line 215	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("site", "ai", "chat API"),

Item	Explanation
Git command at line 227	Repository state, publishing, branch checks, or release automation. Evidence: ("system", "Filesystem, Git,\ncompilers, updater", 1045, 250, 300, 140, "#DCFCE7"),
Installer at line 269	Windows setup, update, shortcut, or installation behavior. Evidence: ("release", "GitHub Release\ninstaller + portable", 1015, 370, 300, 130, "#EDE9FE"),
Compiler or simulator at line 287	Part of the Compile Code workspace or language/tool detection. Evidence: ("lang", "Language profile\nC, C++, Java,\nPython, JS, HDL", 705, 150, 260, 145, "#FEF3C7"),
Security or auth at line 289	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("scope", "HDL testbench\n+ signal scope", 1030, 315, 250, 125, "#DCFCE7"),

JSON/YAML-style keys detected

Item	Explanation
.github/workflows/build-windows-builder.yml	Line 44: ".github/workflows/build-windows-builder.yml": "GitHub Actions workflow that builds the Windows installer and portable application, checks the stable download link, enforces releas
.github/workflows/main-branch-gate.yml	Line 45: ".github/workflows/main-branch-gate.yml": "GitHub Actions guard that rejects direct pushes to main and rejects pull requests into main unless the source branch is development.",
.gitignore	Line 46: ".gitignore": "Git ignore rules for build outputs, temporary files, and local-only data that should not be committed.",
.nojekyll	Line 47: ".nojekyll": "Tells GitHub Pages not to process the site with Jekyll, so folders and static files publish exactly as generated.",
.well-known/security.txt	Line 48: ".well-known/security.txt": "Public security contact file used by browsers, researchers, and scanners to find responsible disclosure information.",
BRANCHING.md	Line 49: "BRANCHING.md": "Human-readable branch model explaining development, main, feature branches, and release flow.",
Backgrounds/.gitkeep	Line 50: "Backgrounds/.gitkeep": "Keeps the Backgrounds folder in Git even when no background image files are present.",
README.md	Line 51: "README.md": "Main README for the standalone builder application: install, updates, workspaces, text editing, publishing, build commands, branch model, and uninstall.",
_headers	Line 52: "_headers": "Cloudflare Pages style HTTP headers for security and caching behavior on the public website.",
assets/apple-touch-icon.png	Line 53: "assets/apple-touch-icon.png": "Apple touch icon shown when the public site is saved on iOS or compatible launch surfaces.",
assets/favicon-16.png	Line 54: "assets/favicon-16.png": "Small browser favicon used by tabs and browser UI.",
assets/favicon-192.png	Line 55: "assets/favicon-192.png": "Large web app icon for Android and installable browser surfaces.",
assets/favicon-32.png	Line 56: "assets/favicon-32.png": "Standard browser favicon size.",
assets/favicon-48.png	Line 57: "assets/favicon-48.png": "Windows/browser icon size used by some desktop surfaces.",
assets/favicon.ico	Line 58: "assets/favicon.ico": "ICO favicon bundle used by browsers that request favicon.ico.",
assets/omb-app-icon-256.png	Line 59: "assets/omb-app-icon-256.png": "Builder application icon source at 256 pixels.",
assets/omb-app-icon-512.png	Line 60: "assets/omb-app-icon-512.png": "Builder application icon source at 512 pixels.",
assets/omb-app-icon.ico	Line 61: "assets/omb-app-icon.ico": "Windows ICO application icon used by Electron Builder and shortcuts.",
assets/omb-mark.png	Line 62: "assets/omb-mark.png": "OMB snake/mark image used for branding in the site and builder.",
build/installer.nsh	Line 63: "build/installer.nsh": "NSIS installer customization script for update behavior, shortcut placement, old install detection, and setup details.",
builder-rich-future-sections.js	Line 64: "builder-rich-future-sections.js": "Builder-side helper code for richer future portfolio sections and section defaults.",
cloudflare/README.md	Line 65: "cloudflare/README.md": "Cloudflare setup notes for deploying the AI Worker endpoint and wiring the public site to the backend.",
cloudflare/portfolio-ai-worker.js	Line 66: "cloudflare/portfolio-ai-worker.js": "Cloudflare Worker backend for the public portfolio AI assistant and fallback intelligence path.",
cloudflare/wrangler.toml	Line 67: "cloudflare/wrangler.toml": "Wrangler configuration used to deploy the Cloudflare Worker.",
docs/.gitkeep	Line 68: "docs/.gitkeep": "Keeps the docs folder in Git even before generated documents are added.",

Item	Explanation
electronics-search.js	Line 69: "electronics-search.js": "Electronics vocabulary and search support used by the website search behavior.",
index.html	Line 70: "index.html": "Public website HTML shell that recruiters and visitors load in the browser.",
installer-notes.txt	Line 71: "installer-notes.txt": "Installer license/notes text shown during Windows setup.",
main.cjs	Line 72: "main.cjs": "Electron main process: creates the desktop window, starts the local backend, handles menus, update handoff, and application lifecycle.",
package.json	Line 73: "package.json": "Node/Electron package manifest with scripts, dependencies, version, installer settings, and extra resources.",
pnpm-lock.yaml	Line 74: "pnpm-lock.yaml": "Pinned dependency lockfile so installs and GitHub Actions use the same dependency versions.",
pnpm-workspace.yaml	Line 75: "pnpm-workspace.yaml": "pnpm workspace declaration for the repository.",
portfolio-README.md	Line 76: "portfolio-README.md": "README for the generated public portfolio website rather than the builder application.",
project-templates.json	Line 77: "project-templates.json": "Appearance template definitions used by projects to control visual feel rather than content.",
projects.json	Line 78: "projects.json": "Public project catalog consumed by the website after parsing builder data.",
robots.txt	Line 79: "robots.txt": "Crawler instruction file for search engines.",
script.js	Line 80: "script.js": "Public website JavaScript for navigation, portfolio rendering, search, AI UI, and interactive project windows.",
server.mjs	Line 81: "server.mjs": "Local backend server used by the builder for drafts, parsing, publishing, compiler execution, authentication checks, and file operations.",
styles.css	Line 82: "styles.css": "Public website CSS for layout, contact area, projects, responsive behavior, AI panel, and mobile/desktop rendering.",
template-preview.css	Line 83: "template-preview.css": "Builder application CSS for dialogs, windows, rich editors, compile code, dark mode, guide windows, and local previews.",
template-preview.html	Line 84: "template-preview.html": "Builder HTML shell loaded inside the Electron app and local preview runtime.",
template-preview.js	Line 85: "template-preview.js": "Builder frontend brain: state handling, rich editors, project builder, previews, parser triggers, publishing UI, compile workspace, and guide windows.",
High-Level System Map	Line 105: "High-Level System Map": "system-overview",
Why Electron Was Chosen	Line 106: "Why Electron Was Chosen": "electron-runtime",
Portfolio Parser Role	Line 107: "Portfolio Parser Role": "builder-to-site-files",
GitHub Pages Publishing	Line 108: "GitHub Pages Publishing": "builder-to-site-files",
Cloudflare Role	Line 109: "Cloudflare Role": "cloudflare-ai-flow",
AI Backend Role	Line 110: "AI Backend Role": "cloudflare-ai-flow",
Compiler Workspace Role	Line 111: "Compiler Workspace Role": "compile-code-flow",
Installer And Update Role	Line 112: "Installer And Update Role": "release-pipeline",
Branch Model	Line 113: "Branch Model": "release-pipeline",
name	Line 196: "name": "system-overview",
title	Line 197: "title": "Private Builder To Public Portfolio",
nodes	Line 198: "nodes": [
edges	Line 208: "edges": [
name	Line 219: "name": "electron-runtime",
title	Line 220: "title": "Electron Runtime Inside The Desktop App",
nodes	Line 221: "nodes": [
edges	Line 229: "edges": [
name	Line 239: "name": "builder-to-site-files",
title	Line 240: "title": "How Builder Edits Become Website Files",

Item	Explanation
nodes	Line 241: "nodes": [
edges	Line 250: "edges": [
name	Line 261: "name": "release-pipeline",
title	Line 262: "title": "Builder Release Pipeline",
nodes	Line 263: "nodes": [
edges	Line 272: "edges": [
name	Line 282: "name": "compile-code-flow",
title	Line 283: "title": "Compile Code Workspace",
nodes	Line 284: "nodes": [
edges	Line 294: "edges": [
name	Line 305: "name": "tool-build-map",
title	Line 306: "title": "What Tool Builds What",
nodes	Line 307: "nodes": [
edges	Line 316: "edges": [
name	Line 327: "name": "file-communication-map",
title	Line 328: "title": "How Key Files Communicate",
nodes	Line 329: "nodes": [
edges	Line 340: "edges": [
name	Line 353: "name": "cloudflare-ai-flow",

Event handlers and user interactions

Item	Explanation
Line 454	"sectionContent.addEventListener('click', async (event) => {\n const button = event.target.closest('[data-compile-run]');\n if (button) await compileActiveFile(project, file);\n}
Line 1531	if ".addEventListener(" in line:

Function-by-function detail

tracked_files - lines 891-895

Item	Explanation
Source location	Lines 891-895 (5 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def tracked_files() -> list[str]:
Touches	Mostly local calculations or local UI state.
Calls detected	run, strip, splitlines
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
891: def tracked_files() -> list[str]:
892:     result = subprocess.run(["git", "ls-files"], cwd=REPO, text=True, capture_output=True, check=True)
```

```
893:     return [line.strip() for line in result.stdout.splitlines() if line.strip()]
894:
895:
```

load_package - lines 896-902

Item	Explanation
Source location	Lines 896-902 (7 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	def load_package() -> dict:
Touches	Mostly local calculations or local UI state.
Calls detected	loads, read_text
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
896: def load_package() -> dict:
897:     try:
898:         return json.loads((REPO / "package.json").read_text(encoding="utf-8"))
899:     except Exception:
900:         return {}
901:
902:
```

load_font - lines 903-910

Item	Explanation
Source location	Lines 903-910 (8 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	def load_font(size: int, bold: bool = False) -> ImageFont.ImageFont:
Touches	Mostly local calculations or local UI state.
Calls detected	Path, exists, truetype, str, load_default
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
903: def load_font(size: int, bold: bool = False) -> ImageFont.ImageFont:
904:     font_name = "arialbd.ttf" if bold else "arial.ttf"
905:     font_path = Path(r"C:\Windows\Fonts") / font_name
906:     if font_path.exists():
907:         return ImageFont.truetype(str(font_path), size=size)
908:     return ImageFont.load_default()
909:
910:
```

wrap_for_box - lines 911-930

Item	Explanation
Source location	Lines 911-930 (20 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def wrap_for_box(label: str, font: ImageFont.ImageFont, max_width: int) -> list[str]:

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	str, splitlines, split, append, getbbox
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
911: def wrap_for_box(label: str, font: ImageFont.ImageFont, max_width: int) -> list[str]:
912:     lines: list[str] = []
913:     for part in str(label).splitlines():
914:         words = part.split()
915:         if not words:
916:             lines.append("")
917:             continue
918:         current = words[0]
919:         for word in words[1:]:
920:             candidate = f"{current} {word}"
921:             bbox = font.getbbox(candidate)
922:             if bbox[2] - bbox[0] <= max_width:
923:                 current = candidate
924:             else:
925:                 lines.append(current)
926:                 current = word
927:         lines.append(current)
928:     return lines
929:
930:
```

draw_arrow - lines 931-943

Item	Explanation
Source location	Lines 931-943 (13 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def draw_arrow(draw: ImageDraw.ImageDraw, start: tuple[int, int], end: tuple[int, int], color: str, width: int = 4) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	line, atan2, int, cos, sin, polygon
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
931: def draw_arrow(draw: ImageDraw.ImageDraw, start: tuple[int, int], end: tuple[int, int], color: str,
width: int = 4) -> None:
932:     draw.line([start, end], fill=color, width=width)
933:     angle = math.atan2(end[1] - start[1], end[0] - start[0])
934:     length = 18
935:     spread = 0.55
936:     points = [
937:         end,
938:         (int(end[0] - length * math.cos(angle - spread)), int(end[1] - length * math.sin(angle -
spread))),
939:         (int(end[0] - length * math.cos(angle + spread)), int(end[1] - length * math.sin(angle +
spread))),
940:     ]
941:     draw.polygon(points, fill=color)
942:
943:
```

edge_points - lines 944-959

Item	Explanation
Source location	Lines 944-959 (16 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def edge_points(source: tuple[int, int, int, int], target: tuple[int, int, int, int]) -> tuple[tuple[int, int], tuple[int, int]]:
Touches	Mostly local calculations or local UI state.
Calls detected	abs
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 944: def edge_points(source: tuple[int, int, int, int], target: tuple[int, int, int, int]) -> tuple[tuple[int,
int], tuple[int, int]]:
 945:     sx1, sy1, sx2, sy2 = source
 946:     tx1, ty1, tx2, ty2 = target
 947:     sc = ((sx1 + sx2) // 2, (sy1 + sy2) // 2)
 948:     tc = ((tx1 + tx2) // 2, (ty1 + ty2) // 2)
 949:     dx = tc[0] - sc[0]
 950:     dy = tc[1] - sc[1]
 951:     if abs(dx) >= abs(dy):
 952:         start = (sx2 if dx >= 0 else sx1, sc[1])
 953:         end = (tx1 if dx >= 0 else tx2, tc[1])
 954:     else:
 955:         start = (sc[0], sy2 if dy >= 0 else sy1)
 956:         end = (tc[0], ty1 if dy >= 0 else ty2)
 957:     return start, end
 958:
 959:
```

make_diagrams - lines 960-1009

Item	Explanation
Source location	Lines 960-1009 (50 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def make_diagrams() -> dict[str, Path]:
Touches	filesystem
Calls detected	mkdir, load_font, Draw, rounded_rectangle, text, line, edge_points, draw_arrow, getbbox, hypot, max, wrap_for_box
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 960: def make_diagrams() -> dict[str, Path]:
 961:     DIAGRAM_DIR.mkdir(parents=True, exist_ok=True)
 962:     title_font = load_font(38, bold=True)
 963:     box_font = load_font(24, bold=True)
 964:     label_font = load_font(18, bold=False)
 965:     diagram_paths: dict[str, Path] = {}
 966:
 967:     for spec in DIAGRAM_SPECS:
 968:         image = Image.new("RGB", (1600, 900), "#F8FBFD")
 969:         draw = ImageDraw.Draw(image)
 970:         draw.rounded_rectangle((24, 24, 1576, 876), radius=26, outline="#B7D7EA", width=3,
fill="#FFFFFF")
 971:         draw.text((60, 48), spec["title"], fill="#0B2545", font=title_font)
 972:         draw.line((60, 105, 1540, 105), fill="#B7D7EA", width=3)
 973:
 974:         boxes: dict[str, tuple[int, int, int, int]] = {}
 975:         for node_id, label, x, y, w, h, fill in spec["nodes"]:
 976:             boxes[node_id] = (x, y, x + w, y + h)
 977:
```

```
    for source_id, target_id, *maybe_label in spec["edges"]:
        source = boxes[source_id]
        target = boxes[target_id]
        start, end = edge_points(source, target)
        draw_arrow(draw, start, end, "#2563EB", width=4)
        if maybe_label:
            label = maybe_label[0]
            mid = ((start[0] + end[0]) // 2, (start[1] + end[1]) // 2)
            text_box = label_font.getbbox(label)
            tw = text_box[2] - text_box[0] + 16
            th = text_box[3] - text_box[1] + 10
            if math.hypot(end[0] - start[0], end[1] - start[1]) > max(tw + 28, 95):
                draw.rounded_rectangle((mid[0] - tw // 2, mid[1] - th // 2, mid[0] + tw // 2, mid[1]
+ th // 2), radius=9, fill="#EFF6FF", outline="#BFDBFE", width=1)
                draw.text((mid[0] - tw // 2 + 8, mid[1] - th // 2 + 4), label, fill="#1E3A8A",
font=label_font)

    for node_id, label, x, y, w, h, fill in spec["nodes"]:
        draw.rounded_rectangle((x, y, x + w, y + h), radius=20, fill=fill, outline="#668EA8",
width=3)
        lines = wrap_for_box(label, box_font, w - 28)
        line_height = 30
        total_h = len(lines) * line_height
        start_y = y + (h - total_h) // 2
        for index, line in enumerate(lines):
            bbox = box_font.getbbox(line)
            tw = bbox[2] - bbox[0]
            draw.text((x + (w - tw) // 2, start_y + index * line_height), line, fill="#102033",
font=box_font)

    path = DIAGRAM_DIR / f"{spec['name']}.png"
    image.save(path)
    diagram_paths[spec["name"]] = path
return diagram_paths
```

add_diagram - lines 1010-1023

Item	Explanation
Source location	Lines 1010-1023 (14 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_diagram(doc: Document, path: Path, caption: str) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	exists, add_paragraph, add_run, add_picture, str, Inches, Pt, RGBColor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
def add_diagram(doc: Document, path: Path, caption: str) -> None:
    if not path.exists():
        return
    paragraph = doc.add_paragraph()
    paragraph.alignment = WD_ALIGN_PARAGRAPH.CENTER
    run = paragraph.add_run()
    run.add_picture(str(path), width=Inches(6.35))
    cap = doc.add_paragraph(caption)
    cap.alignment = WD_ALIGN_PARAGRAPH.CENTER
    for run in cap.runs:
        run.font.size = Pt(9)
        run.font.color.rgb = RGBColor(85, 85, 85)
```

shade - lines 1024-1030

Item	Explanation
Source location	Lines 1024-1030 (7 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.

Item	Explanation
Declaration	def shade(cell, fill: str) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	get_or_add_tcPr, OxmlElement, set, qn, append
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1024: def shade(cell, fill: str) -> None:
1025:     tc_pr = cell._tc.get_or_add_tcPr()
1026:     shd = OxmlElement("w:shd")
1027:     shd.set(qn("w:fill"), fill)
1028:     tc_pr.append(shd)
1029:
1030:
```

cell_text - lines 1031-1039

Item	Explanation
Source location	Lines 1031-1039 (9 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def cell_text(cell, text: str, bold: bool = False) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_run, str, Pt
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1031: def cell_text(cell, text: str, bold: bool = False) -> None:
1032:     cell.text = ""
1033:     run = cell.paragraphs[0].add_run(str(text))
1034:     run.bold = bold
1035:     run.font.name = "Calibri"
1036:     run.font.size = Pt(9)
1037:     cell.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
1038:
1039:
```

add_table - lines 1040-1056

Item	Explanation
Source location	Lines 1040-1056 (17 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_table(doc: Document, rows: list[tuple[str, str]]) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	Inches, cell_text, shade, add_row
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1040: def add_table(doc: Document, rows: list[tuple[str, str]]) -> None:
1041:     table = doc.add_table(rows=1, cols=2)
1042:     table.alignment = WD_TABLE_ALIGNMENT.CENTER
1043:     table.autofit = False
1044:     table.columns[0].width = Inches(1.55)
1045:     table.columns[1].width = Inches(4.95)
1046:     head = table.rows[0].cells
1047:     cell_text(head[0], "Item", True)
1048:     cell_text(head[1], "Explanation", True)
1049:     shade(head[0], "E8EEF5")
1050:     shade(head[1], "E8EEF5")
1051:     for label, value in rows:
1052:         cells = table.add_row().cells
1053:         cell_text(cells[0], label, True)
1054:         cell_text(cells[1], value, False)
1055:
1056:
```

add_code - lines 1057-1064

Item	Explanation
Source location	Lines 1057-1064 (8 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_code(doc: Document, text: str) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_paragraph, add_run, rstrip, set, qn
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1057: def add_code(doc: Document, text: str) -> None:
1058:     p = doc.add_paragraph()
1059:     p.style = doc.styles["CodeBlock"]
1060:     run = p.add_run((text or "").rstrip())
1061:     run.font.name = "Consolas"
1062:     run._element.rPr.rFonts.set(qn("w:eastAsia"), "Consolas")
1063:
1064:
```

add_note - lines 1065-1079

Item	Explanation
Source location	Lines 1065-1079 (15 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_note(doc: Document, title: str, body: str) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_table, Inches, cell, shade, add_run, RGBColor, Pt, add_paragraph
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1065: def add_note(doc: Document, title: str, body: str) -> None:
1066:     table = doc.add_table(rows=1, cols=1)
1067:     table.alignment = WD_TABLE_ALIGNMENT.CENTER
1068:     table.autofit = False
1069:     table.columns[0].width = Inches(6.35)
1070:     cell = table.cell(0, 0)
1071:     shade(cell, "F4F8FC")
1072:     title_run = cell.paragraphs[0].add_run(title)
1073:     title_run.bold = True
1074:     title_run.font.color.rgb = RGBColor(31, 77, 120)
1075:     title_run.font.size = Pt(10)
1076:     paragraph = cell.add_paragraph(body)
1077:     paragraph.style = "Body Text"
1078:
1079:
```

bullets - lines 1080-1084

Item	Explanation
Source location	Lines 1080-1084 (5 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def bullets(doc: Document, items: list[str]) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_paragraph
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1080: def bullets(doc: Document, items: list[str]) -> None:
1081:     for item in items:
1082:         doc.add_paragraph(item, style="List Bullet")
1083:
1084:
```

numbers - lines 1085-1089

Item	Explanation
Source location	Lines 1085-1089 (5 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def numbers(doc: Document, items: list[str]) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_paragraph
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1085: def numbers(doc: Document, items: list[str]) -> None:
1086:     for item in items:
1087:         doc.add_paragraph(item, style="List Number")
1088:
1089:
```

setup_document - lines 1090-1139

Item	Explanation
Source location	Lines 1090-1139 (50 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def setup_document() -> Document:
Touches	filesystem
Calls detected	Document, Inches, set, qn, Pt, from_string, add_style, RGBColor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1090: def setup_document() -> Document:
1091:     doc = Document()
1092:     section = doc.sections[0]
1093:     section.page_height = Inches(11)
1094:     section.page_width = Inches(8.5)
1095:     for section in doc.sections:
1096:         section.top_margin = Inches(0.65)
1097:         section.bottom_margin = Inches(0.6)
1098:         section.left_margin = Inches(0.65)
1099:         section.right_margin = Inches(0.65)
1100:         section.header_distance = Inches(0.28)
1101:         section.footer_distance = Inches(0.28)
1102:
1103:     normal = doc.styles["Normal"]
1104:     normal.font.name = "Calibri"
1105:     normal._element.rPr.rFonts.set(qn("w:eastAsia"), "Calibri")
1106:     normal.font.size = Pt(10)
1107:     normal.paragraph_format.space_after = Pt(3)
1108:     normal.paragraph_format.line_spacing = 1.08
1109:
1110:     for name, size, color, before, after in [
1111:         ("Heading 1", 14, "2E74B5", 9, 4),
1112:         ("Heading 2", 12, "2E74B5", 7, 3),
1113:         ("Heading 3", 10.5, "1F4D78", 5, 2),
1114:     ]:
1115:         style = doc.styles[name]
1116:         style.font.name = "Calibri"
1117:         style._element.rPr.rFonts.set(qn("w:eastAsia"), "Calibri")
1118:         style.font.size = Pt(size)
1119:         style.font.color.rgb = RGBColor.from_string(color)
1120:         style.paragraph_format.space_before = Pt(before)
1121:         style.paragraph_format.space_after = Pt(after)
1122:
1123:     code = doc.styles.add_style("CodeBlock", 1)
1124:     code.font.name = "Consolas"
1125:     code._element.rPr.rFonts.set(qn("w:eastAsia"), "Consolas")
1126:     code.font.size = Pt(7.2)
1127:     code.paragraph_format.space_before = Pt(2)
1128:     code.paragraph_format.space_after = Pt(3)
1129:     code.paragraph_format.line_spacing = 1.0
1130:
1131:     header = doc.sections[0].header.paragraphs[0]
1132:     header.text = "OMB Portfolio Builder and Website Complete Guide"
1133:     header.alignment = WD_ALIGN_PARAGRAPH.RIGHT
1134:     for run in header.runs:
1135:         run.font.size = Pt(8)
1136:         run.font.color.rgb = RGBColor(85, 85, 85)
1137:     return doc
1138:
1139:
```

add_cover - lines 1140-1157

Item	Explanation
Source location	Lines 1140-1157 (18 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_cover(doc: Document, package: dict) -> None:
Touches	filesystem

Item	Explanation
Calls detected	add_paragraph, add_run, Pt, RGBColor, get, add_note, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1140: def add_cover(doc: Document, package: dict) -> None:
 1141:     title = doc.add_paragraph()
 1142:     title.alignment = WD_ALIGN_PARAGRAPH.CENTER
 1143:     run = title.add_run("OMB Portfolio Builder and Website\nComplete Creation Guide")
 1144:     run.bold = True
 1145:     run.font.size = Pt(26)
 1146:     run.font.color.rgb = RGBColor(11, 37, 69)
 1147:     subtitle = doc.add_paragraph()
 1148:     subtitle.alignment = WD_ALIGN_PARAGRAPH.CENTER
 1149:     r = subtitle.add_run("A beginner-friendly manual explaining the desktop app, public website, GitHub
workflows, installer, parser, compiler tools, AI backend, and publishing system.")
 1150:     r.font.size = Pt(12)
 1151:     meta = doc.add_paragraph()
 1152:     meta.alignment = WD_ALIGN_PARAGRAPH.CENTER
 1153:     meta.add_run(f"Repository: otienomaurice/omb-portfolio-builder\nVersion documented:
{package.get('version', 'unknown')}\nGenerated: July 4, 2026")
 1154:     add_note(doc, "How this guide is written", "This guide starts from the highest-level idea and then
drills down. It intentionally explains terms that engineers usually skip: what Electron is, what a workflow is,
what a local backend is, why Git branches exist, and why the builder and website are separated.")
 1155:     doc.add_page_break()
 1156:
 1157:
```

add_reading_map - lines 1158-1171

Item	Explanation
Source location	Lines 1158-1171 (14 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_reading_map(doc: Document) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_heading, add_paragraph, add_table, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1158: def add_reading_map(doc: Document) -> None:
 1159:     doc.add_heading("Reading Map", level=1)
 1160:     doc.add_paragraph("Use this manual in layers. First read the high-level design pages. Then read the
app flow pages. After that, use the file-by-file and command pages as a reference when you need to understand a
specific file or command.")
 1161:     add_table(doc, [
 1162:         ("Part 1", "High-level architecture and big-picture mental model."),
 1163:         ("Part 2", "Builder app internals: Electron, local backend, parser, rich editors, previews,
compile code, and publishing."),
 1164:         ("Part 3", "Public website internals: HTML, CSS, JavaScript, projects.json, assets, search, AI,
and mobile behavior."),
 1165:         ("Part 4", "GitHub, branches, workflows, releases, installers, and updates."),
 1166:         ("Part 5", "Every tracked file explained in plain English."),
 1167:         ("Part 6", "Commands, tradeoffs, troubleshooting, and safe operating procedures."),
 1168:     ])
 1169:     doc.add_page_break()
 1170:
 1171:
```

add_entry_points - lines 1172-1188

Item	Explanation
Source location	Lines 1172-1188 (17 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_entry_points(doc: Document) -> None:
Touches	filesystem
Calls detected	add_heading, add_paragraph, add_table, numbers, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1172: def add_entry_points(doc: Document) -> None:
 1173:     doc.add_heading("Where Someone Starts: Entry Points", level=1)
 1174:     doc.add_paragraph("An entry point is the first file, executable, or function that starts a flow. This
 project has several entry points because normal users, developers, GitHub Actions, Cloudflare, and installers
all start in different places.")
 1175:     add_table(doc, [(name, f"{entry}. {meaning}") for name, entry, meaning in ENTRY_POINTS])
 1176:     doc.add_heading("Fastest way to orient yourself", level=2)
 1177:     numbers(doc, [
 1178:         "If you are using the installed app, start with OMB Portfolio Builder.exe.",
 1179:         "If you are debugging desktop startup, start with package.json and main.cjs.",
 1180:         "If you are debugging local APIs, start with server.mjs.",
 1181:         "If you are debugging builder UI, start with template-preview.html, template-preview.js, and
template-preview.css.",
 1182:         "If you are debugging the public website, start with index.html, script.js, styles.css, and
projects.json.",
 1183:         "If you are debugging installer behavior, start with package.json build.nsis and
build/installer.nsh.",
 1184:         "If you are debugging releases, start with .github/workflows/build-windows-builder.yml.",
 1185:     ])
 1186:     doc.add_page_break()
 1187:
 1188:
```

add_flow_walkthroughs - lines 1189-1199

Item	Explanation
Source location	Lines 1189-1199 (11 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_flow_walkthroughs(doc: Document, diagrams: dict[str, Path]) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_heading, add_diagram, add_paragraph, add_table, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1189: def add_flow_walkthroughs(doc: Document, diagrams: dict[str, Path]) -> None:
 1190:     for title, diagram_name, steps, debug_note in FLOW_WALKTHROUGHS:
 1191:         doc.add_heading(f"Detailed Walkthrough: {title}", level=1)
 1192:         add_diagram(doc, diagrams[diagram_name], f"Context diagram for {title}.")
 1193:         doc.add_paragraph("This walkthrough follows the real direction of work through the system. Read
it slowly: each line is one handoff from one layer to another.")
 1194:         add_table(doc, steps)
 1195:         doc.add_heading("Debug note", level=2)
 1196:         doc.add_paragraph(debug_note)
 1197:         doc.add_page_break()
 1198:
 1199:
```

add_data_contracts - lines 1200-1212

Item	Explanation
Source location	Lines 1200-1212 (13 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_data_contracts(doc: Document) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_heading, add_paragraph, add_code, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1200: def add_data_contracts(doc: Document) -> None:
1201:     for title, purpose, example, note in DATA_CONTRACTS:
1202:         doc.add_heading(f"Data Contract: {title}", level=1)
1203:         doc.add_paragraph(purpose)
1204:         doc.add_heading("Example shape", level=2)
1205:         add_code(doc, example)
1206:         doc.add_heading("How to read this", level=2)
1207:         doc.add_paragraph(note)
1208:         doc.add_heading("Why contracts matter", level=2)
1209:         doc.add_paragraph("A data contract is an agreement between two pieces of software. If the
frontend sends one shape and the backend expects another shape, the feature breaks even if both files are
individually valid code.")
1210:         doc.add_page_break()
1211:
1212:
```

endpoint_purpose - lines 1213-1242

Item	Explanation
Source location	Lines 1213-1242 (30 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def endpoint_purpose(endpoint: str) -> str:
Touches	filesystem, authentication/security data
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 14 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1213: def endpoint_purpose(endpoint: str) -> str:
1214:     if "code/compile" in endpoint:
1215:         return "Compiles or runs a source file and returns terminal output."
1216:     if "code/save" in endpoint:
1217:         return "Saves source code into the local compile workspace."
1218:     if "code/beautify" in endpoint:
1219:         return "Formats source code for cleaner display and editing."
1220:     if "code/tools" in endpoint:
1221:         return "Checks compiler/runtime availability."
1222:     if "code/install" in endpoint:
1223:         return "Attempts to install missing compiler tools."
1224:     if "publish-target" in endpoint:
1225:         return "Reads or writes the Git publishing target and authorization state."
1226:     if "publish" in endpoint:
1227:         return "Publishes generated website files to the target repository."
1228:     if "catalog" in endpoint:
1229:         return "Loads project/catalog data for the builder."
1230:     if "templates" in endpoint:
```

```
 1231:         return "Loads appearance template definitions."
 1232:     if "app-update" in endpoint:
 1233:         return "Checks or starts builder app update behavior."
 1234:     if "security-report" in endpoint:
 1235:         return "Returns security/download/auth reporting for the builder."
 1236:     if "portfolio-ai" in endpoint:
 1237:         return "Handles AI assistant questions."
 1238:     if "system-check" in endpoint:
 1239:         return "Checks local tool/system readiness."
 1240:     return "Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact
behavior."
 1241:
 1242:
```

extract_api_endpoints - lines 1243-1255

Item	Explanation
Source location	Lines 1243-1255 (13 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def extract_api_endpoints() -> list[tuple[str, str, str]]:
Touches	network/API requests
Calls detected	exists, enumerate, read_text, splitlines, findall, rstrip, min, get, str, sorted, items
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1243: def extract_api_endpoints() -> list[tuple[str, str, str]]:
 1244:     endpoints: dict[tuple[str, str], int] = {}
 1245:     for file_name in ["template-preview.js", "script.js", "server.mjs", "index.html"]:
 1246:         path = REPO / file_name
 1247:         if not path.exists():
 1248:             continue
 1249:         for line_no, line in enumerate(path.read_text(encoding="utf-8", errors="replace").splitlines(),
start=1):
 1250:             for match in re.findall(r"['\"](/api/[A-Za-z0-9_./-]+)", line):
 1251:                 clean = match.rstrip("/")
 1252:                 endpoints[(clean, file_name)] = min(line_no, endpoints.get((clean, file_name), line_no))
 1253:     return [(endpoint, file_name, str(line_no)) for (endpoint, file_name), line_no in
sorted(endpoints.items())]
 1254:
 1255:
```

add_api_endpoint_catalog - lines 1256-1278

Item	Explanation
Source location	Lines 1256-1278 (23 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_api_endpoint_catalog(doc: Document) -> None:
Touches	filesystem
Calls detected	extract_api_endpoints, add_heading, add_paragraph, add_page_break, endpoint_purpose, add_table
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1256: def add_api_endpoint_catalog(doc: Document) -> None:
 1257:     endpoints = extract_api_endpoints()
 1258:     doc.add_heading("API Endpoint Catalog", level=1)
 1259:     doc.add_paragraph("This catalog is generated from strings found in the source files. It tells you
```

```
which /api paths are used and where to start reading when a request fails.")
 1260:     if not endpoints:
 1261:         doc.add_paragraph("No /api endpoints were found by the scanner.")
 1262:         doc.add_page_break()
 1263:         return
 1264:     rows = [(endpoint, f"Seen in {file_name} near line {line_no}. Purpose: {endpoint_purpose(endpoint)}")
 for endpoint, file_name, line_no in endpoints]
 1265:     add_table(doc, rows)
 1266:     doc.add_page_break()
 1267:     for endpoint, file_name, line_no in endpoints:
 1268:         doc.add_heading(f"API Detail: {endpoint}", level=1)
 1269:         add_table(doc, [
 1270:             ("Where seen", f"{file_name} near line {line_no}"),
 1271:             ("Purpose", endpoint_purpose(endpoint)),
 1272:             ("Frontend question", "Which button, form, or page action sends this request?"),
 1273:             ("Backend question", "Which handler in server.mjs receives this path and what files/tools
does it touch?"),
 1274:             ("Debugging", "Check browser network status, response JSON, server logs, and whether stale
cache was avoided with no-store or a timestamp."),
 1275:         ])
 1276:         doc.add_page_break()
 1277:
 1278:
```

function_purpose - lines 1279-1317

Item	Explanation
Source location	Lines 1279-1317 (39 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def function_purpose(file_name: str, function_name: str) -> str:
Touches	filesystem, authentication/security data
Calls detected	lower, startswith
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 18 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1279: def function_purpose(file_name: str, function_name: str) -> str:
 1280:     name = function_name.lower()
 1281:     if name.startswith("render"):
 1282:         return "Renders UI, markup, or display output."
 1283:     if name.startswith("handle"):
 1284:         return "Handles an event, request, command, or user action."
 1285:     if name.startswith("update"):
 1286:         return "Updates UI, state, cache, or stored data."
 1287:     if name.startswith("save") or name.startswith("write") or name.startswith("store"):
 1288:         return "Persists data to local state, disk, credentials, or browser storage."
 1289:     if name.startswith("read") or name.startswith("load") or name.startswith("fetch"):
 1290:         return "Loads data from disk, network, browser storage, or a service."
 1291:     if "compile" in name or "compiler" in name:
 1292:         return "Part of the Compile Code language/tool/build/run flow."
 1293:     if "publish" in name or "git" in name or "remote" in name:
 1294:         return "Part of publishing, Git, target repository, or authorization behavior."
 1295:     if "auth" in name or "credential" in name or "access" in name:
 1296:         return "Part of authentication, authorization, or credential checking."
 1297:     if "cache" in name:
 1298:         return "Reads, writes, or validates cached state."
 1299:     if name.startswith("validate") or name.startswith("normalize") or name.startswith("safe") or
name.startswith("sanitize"):
 1300:         return "Validates or normalizes input so later code receives safe predictable values."
 1301:     if name.startswith("create") or name.startswith("new") or name.startswith("build"):
 1302:         return "Creates an object, UI structure, process, payload, or generated artifact."
 1303:     if name.startswith("open") or name.startswith("close") or name.startswith("show") or
name.startswith("hide"):
 1304:         return "Controls a window, dialog, panel, menu, or visible state."
 1305:     if name.startswith("sync"):
 1306:         return "Keeps two places aligned, such as local data and target files."
 1307:     if file_name == "main.cjs":
 1308:         return "Part of Electron desktop startup, window control, workspace setup, menu behavior, or
update handoff."
 1309:     if file_name == "server.mjs":
 1310:         return "Part of the local backend API, filesystem, compiler, Git, AI, update, or publish
workflow."
 1311:     if file_name == "template-preview.js":
 1312:         return "Part of the private builder frontend and editing experience."
 1313:     if file_name == "script.js":
```

```
1314:         return "Part of the public website runtime."
1315:     return "Named function in the repository. Inspect the surrounding lines to learn its exact inputs and
outputs."
1316:
1317:
```

line_excerpt - lines 1352-1361

Item	Explanation
Source location	Lines 1352-1361 (10 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def line_excerpt(lines: list[str], start: int, end: int, max_lines: int = 120) -> str:
Touches	Mostly local calculations or local UI state.
Calls detected	max, min, len, enumerate, append, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1352: def line_excerpt(lines: list[str], start: int, end: int, max_lines: int = 120) -> str:
1353:     selected = lines[max(start - 1, 0): min(end, len(lines))]
1354:     truncated = len(selected) > max_lines
1355:     selected = selected[:max_lines]
1356:     rendered = [f"{start + index:>5}: {line}" for index, line in enumerate(selected)]
1357:     if truncated:
1358:         rendered.append(f"      ... excerpt truncated after {max_lines} lines. Open the source file for
the rest of this function.")
1359:     return "\n".join(rendered)
1360:
1361:
```

find_block_end - lines 1362-1377

Item	Explanation
Source location	Lines 1362-1377 (16 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def find_block_end(lines: list[str], start_index: int) -> int:
Touches	Mostly local calculations or local UI state.
Calls detected	range, min, len, sub, count, startswith
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1362: def find_block_end(lines: list[str], start_index: int) -> int:
1363:     brace_balance = 0
1364:     saw_brace = False
1365:     for index in range(start_index, min(start_index + 500, len(lines))):
1366:         line = re.sub(r"//.*$", "", lines[index])
1367:         brace_balance += line.count("{")
1368:         brace_balance -= line.count("}")
1369:         if "{" in line:
1370:             saw_brace = True
1371:         if saw_brace and brace_balance <= 0 and index > start_index:
1372:             return index + 1
1373:         if not saw_brace and index > start_index and not lines[index].startswith((" ", "\t")):
1374:             return index
1375:     return min(start_index + 80, len(lines))
1376:
1377:
```

extract_function_blocks - lines 1378-1423

Item	Explanation
Source location	Lines 1378-1423 (46 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def extract_function_blocks(repo_file: str, lines: list[str]) -> list[dict]:
Touches	browser/Electron DOM, filesystem, network/API requests, authentication/security data
Calls detected	enumerate, strip, search, group, lstrip, startswith, next, range, len, min, find_block_end, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1378: def extract_function_blocks(repo_file: str, lines: list[str]) -> list[dict]:
1379:     functions = []
1380:     for index, line in enumerate(lines):
1381:         stripped = line.strip()
1382:         for pattern in FUNCTION_PATTERNS:
1383:             match = pattern.search(line)
1384:             if not match:
1385:                 continue
1386:             name = match.group(1)
1387:             if line.lstrip().startswith("Function "):
1388:                 end = next((i + 1 for i in range(index + 1, len(lines)) if
lines[i].lstrip().startswith("FunctionEnd")), min(index + 80, len(lines)))
1389:             elif line.lstrip().startswith("def "):
1390:                 end = next((i for i in range(index + 1, len(lines)) if lines[i] and not
lines[i].startswith((" ", "\t", "@"))), min(index + 120, len(lines)))
1391:             else:
1392:                 end = find_block_end(lines, index)
1393:             body = "\n".join(lines[index:end])
1394:             call_candidates = re.findall(r"\b([A-Za-z_$][\w$]*)\s*\(", body)
1395:             calls = []
1396:             for candidate in call_candidates:
1397:                 if candidate not in COMMON_CALL_WORDS and candidate != name and candidate not in calls:
1398:                     calls.append(candidate)
1399:             endpoint_hits = sorted(set(match.rstrip("/") for match in
re.findall(r"['\"](/api/[A-Za-z0-9_./-]+)", body)))
1400:             local_storage_keys = sorted(set(match for match in
re.findall(r"(?:localStorage|sessionStorage)\.(?:getItem|setItem|removeItem)\(['\"]([^'\"]+)", body)))
1401:             functions.append({
1402:                 "line": index + 1,
1403:                 "end_line": end,
1404:                 "name": name,
1405:                 "purpose": function_purpose(Path(repo_file).name, name),
1406:                 "signature": stripped[:240],
1407:                 "excerpt": line_excerpt(lines, index + 1, end, max_lines=120),
1408:                 "line_count": max(end - index, 1),
1409:                 "calls": calls[:30],
1410:                 "endpoints": endpoint_hits[:30],
1411:                 "storage_keys": local_storage_keys[:30],
1412:                 "returns": len(re.findall(r"\breturn\b", body)),
1413:                 "awaits": len(re.findall(r"\bawait\b", body)),
1414:                 "touches_dom":
bool(re.search(r"document\.|querySelector|getElementById|classList|dataset", body)),
1415:                 "touches_files":
bool(re.search(r"readFile|writeFile|appendFile|copyFile|mkdir|rm|unlink|fs\.", body, re.IGNORECASE)),
1416:                 "runs_process": bool(re.search(r"\bspawn\b|\bexecFile\b|\bexec\b", body, re.IGNORECASE)),
1417:                 "uses_network": bool(re.search(r"\bfetch\s*\(|https?://|/api/", body, re.IGNORECASE)),
1418:                 "uses_security":
bool(re.search(r"auth|token|secret|credential|permission|authorize|scope", body, re.IGNORECASE)),
1419:             })
1420:             break
1421:     return functions
1422:
1423:
```

collect_signal_hits - lines 1424-1441

Item	Explanation
Source location	Lines 1424-1441 (18 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.

Item	Explanation
Declaration	def collect_signal_hits(lines: list[str]) -> list[dict]:
Touches	Mostly local calculations or local UI state.
Calls detected	enumerate, strip, search, append
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1424: def collect_signal_hits(lines: list[str]) -> list[dict]:
1425:     hits = []
1426:     for line_no, line in enumerate(lines, start=1):
1427:         stripped = line.strip()
1428:         if not stripped:
1429:             continue
1430:         for name, pattern, explanation in SOURCE_SIGNAL_RULES:
1431:             if pattern.search(line):
1432:                 hits.append({
1433:                     "line": line_no,
1434:                     "type": name,
1435:                     "evidence": stripped[:220],
1436:                     "meaning": explanation,
1437:                 })
1438:                 break
1439:     return hits
1440:
1441:
```

function_detail_rows - lines 1442-1466

Item	Explanation
Source location	Lines 1442-1466 (25 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def function_detail_rows(item: dict) -> list[tuple[str, str]]:
Touches	authentication/security data
Calls detected	append, join, expression, statement
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1442: def function_detail_rows(item: dict) -> list[tuple[str, str]]:
1443:     touches = []
1444:     if item["touches_dom"]:
1445:         touches.append("browser/Electron DOM")
1446:     if item["touches_files"]:
1447:         touches.append("filesystem")
1448:     if item["runs_process"]:
1449:         touches.append("external processes")
1450:     if item["uses_network"]:
1451:         touches.append("network/API requests")
1452:     if item["uses_security"]:
1453:         touches.append("authentication/security data")
1454:     return [
1455:         ("Source location", f"Lines {item['line']}-{item['end_line']} ({item['line_count']} source
lines)."),
1456:         ("Purpose", item["purpose"]),
1457:         ("Declaration", item["signature"]),
1458:         ("Touches", ", ".join(touches) if touches else "Mostly local calculations or local UI state."),
1459:         ("Calls detected", ", ".join(item["calls"][:12]) if item["calls"] else "No obvious named calls
detected by the generator."),
1460:         ("API endpoints", ", ".join(item["endpoints"]) if item["endpoints"] else "None detected inside
this function body."),
```

```
 1461:         ("Storage keys", ", ".join(item["storage_keys"]) if item["storage_keys"] else "No local/session
storage keys detected."),
 1462:         ("Async/return clues", f"{item['awaits']} await expression(s), {item['returns']} return
statement(s)."),
 1463:         ("Debug approach", "Trigger the user action that reaches this function, inspect inputs/state
before the function, then inspect the returned value, DOM update, file write, process output, or API response
after it runs."),
 1464:     ]
 1465:
 1466:
```

extract_functions - lines 1467-1482

Item	Explanation
Source location	Lines 1467-1482 (16 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def extract_functions() -> list[tuple[str, int, str, str]]:
Touches	Mostly local calculations or local UI state.
Calls detected	exists, enumerate, read_text, splitlines, search, group, append, function_purpose
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1467: def extract_functions() -> list[tuple[str, int, str, str]]:
 1468:     function_rows: list[tuple[str, int, str, str]] = []
 1469:     for file_name in ["main.cjs", "server.mjs", "template-preview.js", "script.js",
"cloudflare/portfolio-ai-worker.js", "docs/build_complete_guide.py", "build/installer.nsh"]:
 1470:         path = REPO / file_name
 1471:         if not path.exists():
 1472:             continue
 1473:         for line_no, line in enumerate(path.read_text(encoding="utf-8", errors="replace").splitlines(),
start=1):
 1474:             for pattern in FUNCTION_PATTERNS:
 1475:                 match = pattern.search(line)
 1476:                 if match:
 1477:                     name = match.group(1)
 1478:                     function_rows.append((file_name, line_no, name, function_purpose(file_name, name)))
 1479:                     break
 1480:     return function_rows
 1481:
 1482:
```

is_text_code_file - lines 1488-1496

Item	Explanation
Source location	Lines 1488-1496 (9 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def is_text_code_file(repo_file: str) -> bool:
Touches	Mostly local calculations or local UI state.
Calls detected	startswith, endswith, is_file, lower
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1488: def is_text_code_file(repo_file: str) -> bool:
 1489:     path = REPO / repo_file
 1490:     if repo_file.startswith(("docs/code-reference/", "docs/code-reference-docx/", "docs/code-reference-
pdf/", "docs/guide-render-check/")):
```

```
1491:         return False
1492:     if repo_file.endswith((".docx", ".pdf", ".png", ".ico")):
1493:         return False
1494:     return path.is_file() and path.suffix.lower() in TEXT_CODE_EXTENSIONS
1495:
1496:
```

source_lines - lines 1497-1503

Item	Explanation
Source location	Lines 1497-1503 (7 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def source_lines(repo_file: str) -> list[str]:
Touches	Mostly local calculations or local UI state.
Calls detected	exists, is_file, read_text, splitlines
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1497: def source_lines(repo_file: str) -> list[str]:
1498:     path = REPO / repo_file
1499:     if not path.exists() or not path.is_file():
1500:         return []
1501:     return path.read_text(encoding="utf-8", errors="replace").splitlines()
1502:
1503:
```

extract_source_facts - lines 1504-1563

Item	Explanation
Source location	Lines 1504-1563 (60 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def extract_source_facts(repo_file: str) -> dict:
Touches	network/API requests
Calls detected	source_lines, join, extract_function_blocks, enumerate, strip, match, len, append, group, findall, rstrip, search
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1504: def extract_source_facts(repo_file: str) -> dict:
1505:     lines = source_lines(repo_file)
1506:     text = "\n".join(lines)
1507:     functions = extract_function_blocks(repo_file, lines)
1508:     variables = []
1509:     imports = []
1510:     endpoints = []
1511:     selectors = []
1512:     events = []
1513:     routes = []
1514:     json_keys = []
1515:     for line_no, line in enumerate(lines, start=1):
1516:         stripped = line.strip()
1517:         var_match = re.match(r"^\s*(?:const|let|var)\s+([A-Za-z_$][\w$]*)\s*=", line)
1518:         if var_match and len(variables) < 140:
1519:             variables.append({
1520:                 "line": line_no,
1521:                 "name": var_match.group(1),
1522:                 "statement": stripped[:180],
```

```
 1523:             })
 1524:         if re.match(r"^\s*import\s+", line) or re.match(r"^\s*(?:const|let|var)\s+\S+\s*=\s*require\(",
line):
 1525:             imports.append({"line": line_no, "statement": stripped[:220]})
 1526:         for match in re.findall(r['\"](/api/[A-Za-z0-9_./-]+)", line):
 1527:             endpoints.append({"line": line_no, "endpoint": match.rstrip("/")})
 1528:         route_match = re.search(r"\bapp\.(get|post|put|delete|patch)\(['\"]([^'\"]+)", line)
 1529:         if route_match:
 1530:             routes.append({"line": line_no, "method": route_match.group(1).upper(), "path":
route_match.group(2)})
 1531:         if ".addEventListener(" in line:
 1532:             events.append({"line": line_no, "statement": stripped[:180]})
 1533:         json_match = re.match(r'^\s*"([^"]+)"\s*:', line)
 1534:         if json_match and len(json_keys) < 120:
 1535:             json_keys.append({"line": line_no, "key": json_match.group(1), "statement": stripped[:180]})
 1536:         if repo_file.endswith(".css"):
 1537:             selector_match = re.match(r"^\s*([.#A-Za-z0-9_\-:\[\]\(\)\s>,+.=\"']+)\s*\{", line)
 1538:             if selector_match and len(selectors) < 140:
 1539:                 selectors.append({"line": line_no, "selector": selector_match.group(1).strip()[:160]})
 1540:     return {
 1541:         "repo_file": repo_file,
 1542:         "line_count": len(lines),
 1543:         "size_bytes": (REPO / repo_file).stat().st_size if (REPO / repo_file).exists() else 0,
 1544:         "description": FILE_DESCRIPTIONS.get(repo_file, "Tracked source/configuration file in the OMB
Portfolio Builder repository."),
 1545:         "functions": functions,
 1546:         "variables": variables,
 1547:         "imports": imports,
 1548:         "endpoints": endpoints,
 1549:         "routes": routes,
 1550:         "selectors": selectors,
 1551:         "events": events,
 1552:         "json_keys": json_keys,
 1553:         "signals": collect_signal_hits(lines),
 1554:         "mentions_server": "server.mjs" in text,
 1555:         "mentions_template_preview": "template-preview" in text,
 1556:         "mentions_script": "script.js" in text,
 1557:         "mentions_projects_json": "projects.json" in text,
 1558:         "mentions_github": "github" in text.lower(),
 1559:         "mentions_cloudflare": "cloudflare" in text.lower() or "worker" in text.lower(),
 1560:         "snippet": line_excerpt(lines, 1, min(72, len(lines)), max_lines=72) if lines else "",
 1561:     }
 1562:
 1563:
```

fact_relationship_summary - lines 1564-1580

Item	Explanation
Source location	Lines 1564-1580 (17 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def fact_relationship_summary(facts: dict) -> str:
Touches	Mostly local calculations or local UI state.
Calls detected	get, append, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1564: def fact_relationship_summary(facts: dict) -> str:
 1565:     related = []
 1566:     if facts.get("mentions_template_preview"):
 1567:         related.append("builder frontend")
 1568:     if facts.get("mentions_server"):
 1569:         related.append("local backend")
 1570:     if facts.get("mentions_script"):
 1571:         related.append("public website runtime")
 1572:     if facts.get("mentions_projects_json"):
 1573:         related.append("portfolio catalog")
 1574:     if facts.get("mentions_cloudflare"):
 1575:         related.append("Cloudflare/AI layer")
 1576:     if facts.get("mentions_github"):
 1577:         related.append("GitHub/release layer")
 1578:     return ", ".join(related) if related else "Mostly local to its own feature area."
 1579:
 1580:
```

code_reference_name - lines 1581-1584

Item	Explanation
Source location	Lines 1581-1584 (4 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def code_reference_name(repo_file: str, suffix: str) -> str:
Touches	Mostly local calculations or local UI state.
Calls detected	sub, strip
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1581: def code_reference_name(repo_file: str, suffix: str) -> str:
1582:     return re.sub(r"[^A-Za-z0-9_.-]+", "__", repo_file).strip("_") + suffix
1583:
1584:
```

remove_directory - lines 1585-1598

Item	Explanation
Source location	Lines 1585-1598 (14 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def remove_directory(path: Path) -> None:
Touches	filesystem
Calls detected	exists, remove_readonly, Path, chmod, rmtree
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1585: def remove_directory(path: Path) -> None:
1586:     if not path.exists():
1587:         return
1588:
1589:     def remove_readonly(function, item_path, excinfo):
1590:         try:
1591:             Path(item_path).chmod(0o700)
1592:             function(item_path)
1593:         except Exception:
1594:             raise excinfo
1595:
1596:     shutil.rmtree(path, onexc=remove_readonly)
1597:
1598:
```

remove_readonly - lines 1589-1598

Item	Explanation
Source location	Lines 1589-1598 (10 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def remove_readonly(function, item_path, excinfo):
Touches	filesystem

Item	Explanation
Calls detected	Path, chmod, rmtree
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1589:     def remove_readonly(function, item_path, excinfo):
1590:         try:
1591:             Path(item_path).chmod(0o700)
1592:             function(item_path)
1593:         except Exception:
1594:             raise excinfo
1595:
1596:     shutil.rmtree(path, onexc=remove_readonly)
1597:
1598:
```

add_source_fact_summary - lines 1599-1612

Item	Explanation
Source location	Lines 1599-1612 (14 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_source_fact_summary(doc: Document, facts: dict) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_table, fact_relationship_summary, str, len
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1599: def add_source_fact_summary(doc: Document, facts: dict) -> None:
1600:     add_table(doc, [
1601:         ("File", facts["repo_file"]),
1602:         ("Purpose", facts["description"]),
1603:         ("Lines", f"{facts['line_count']:,}"),
1604:         ("Size", f"{facts['size_bytes']:,} bytes"),
1605:         ("Talks to", fact_relationship_summary(facts)),
1606:         ("Functions", str(len(facts["functions"]))),
1607:         ("Variables/constants", str(len(facts["variables"]))),
1608:         ("API mentions", str(len(facts["endpoints"]))),
1609:         ("Signals", str(len(facts["signals"]))),
1610:     ])
1611:
1612:
```

add_source_fact_sections - lines 1613-1659

Item	Explanation
Source location	Lines 1613-1659 (47 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_source_fact_sections(doc: Document, facts: dict, include_excerpts: bool = True) -> None:
Touches	filesystem
Calls detected	add_heading, add_paragraph, add_table, endpoint_purpose, function_detail_rows, add_code, numbers
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1613: def add_source_fact_sections(doc: Document, facts: dict, include_excerpts: bool = True) -> None:
 1614:     doc.add_heading("How this file participates in the system", level=2)
 1615:     doc.add_paragraph("This generated section reads the actual file and points out how it communicates
with other pieces of the app. It is intentionally concrete: line numbers, declarations, endpoints, events,
source excerpts, and debugging cues.")
 1616:     if facts["imports"]:
 1617:         doc.add_heading("Imports, requires, and external dependencies", level=2)
 1618:         add_table(doc, [(f"Line {item['line']}", item["statement"]) for item in facts["imports"][:60]])
 1619:     if facts["routes"]:
 1620:         doc.add_heading("Backend routes implemented here", level=2)
 1621:         add_table(doc, [(f"{item['method']} {item['path']}", f"Line {item['line']}.
{endpoint_purpose(item['path'])}") for item in facts["routes"][:80]])
 1622:     if facts["endpoints"]:
 1623:         doc.add_heading("API endpoints mentioned or called", level=2)
 1624:         add_table(doc, [(item["endpoint"], f"Line {item['line']}. {endpoint_purpose(item['endpoint'])}")
for item in facts["endpoints"][:80]])
 1625:     if facts["signals"]:
 1626:         doc.add_heading("Important implementation signals", level=2)
 1627:         add_table(doc, [(f"{item['type']} at line {item['line']}", f"{item['meaning']} Evidence:
{item['evidence']}") for item in facts["signals"][:80]])
 1628:     if facts["variables"]:
 1629:         doc.add_heading("Important constants and variables", level=2)
 1630:         add_table(doc, [(item["name"], f"Line {item['line']}: {item['statement']}") for item in
facts["variables"][:80]])
 1631:     if facts["json_keys"]:
 1632:         doc.add_heading("JSON/YAML-style keys detected", level=2)
 1633:         add_table(doc, [(item["key"], f"Line {item['line']}: {item['statement']}") for item in
facts["json_keys"][:80]])
 1634:     if facts["events"]:
 1635:         doc.add_heading("Event handlers and user interactions", level=2)
 1636:         add_table(doc, [(f"Line {item['line']}", item["statement"]) for item in facts["events"][:80]])
 1637:     if facts["selectors"]:
 1638:         doc.add_heading("CSS selectors and visual hooks", level=2)
 1639:         add_table(doc, [(f"Line {item['line']}", item["selector"]) for item in facts["selectors"][:120]])
 1640:     if facts["functions"]:
 1641:         doc.add_heading("Function-by-function detail", level=2)
 1642:         for item in facts["functions"]:
 1643:             doc.add_heading(f"{item['name']} - lines {item['line']}-{item['end_line']}", level=3)
 1644:             add_table(doc, function_detail_rows(item))
 1645:             if include_excerpts:
 1646:                 doc.add_paragraph("Source excerpt:")
 1647:                 add_code(doc, item["excerpt"])
 1648:     doc.add_heading("Representative opening snippet", level=2)
 1649:     add_code(doc, facts["snippet"] or "(empty file)")
 1650:     doc.add_heading("Beginner debugging checklist for this file", level=2)
 1651:     numbers(doc, [
 1652:         "Name the user action, command, or workflow that reaches this file.",
 1653:         "Find the exact function or route that owns the behavior.",
 1654:         "Check the inputs: DOM state, request body, file path, local storage value, compiler source, or
Git branch.",
 1655:         "Check the outputs: returned JSON, updated DOM, written file, terminal output, generated project
catalog, or published website asset.",
 1656:         "If the issue crosses a boundary, test the boundary directly: browser console for frontend,
endpoint call for backend, command line for Git/compiler, Cloudflare logs for Worker behavior.",
 1657:     ])
 1658:
 1659:
```

setup_code_reference_document - lines 1660-1673

Item	Explanation
Source location	Lines 1660-1673 (14 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def setup_code_reference_document(title: str, subtitle: str) -> Document:
Touches	Mostly local calculations or local UI state.
Calls detected	setup_document, add_paragraph, add_run, Pt, RGBColor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1660: def setup_code_reference_document(title: str, subtitle: str) -> Document:
1661:     doc = setup_document()
1662:     title_p = doc.add_paragraph()
1663:     title_p.alignment = WD_ALIGN_PARAGRAPH.LEFT
1664:     run = title_p.add_run(title)
1665:     run.bold = True
1666:     run.font.size = Pt(22)
1667:     run.font.color.rgb = RGBColor(11, 37, 69)
1668:     doc.add_paragraph(subtitle)
1669:     doc.add_paragraph(f"Generated from repository state at {REPO}.")
1670:     doc.add_paragraph("")
1671:     return doc
1672:
1673:
```

compact_generated_docx - lines 1674-1694

Item	Explanation
Source location	Lines 1674-1694 (21 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def compact_generated_docx(doc: Document, keep_first_page_break: bool = True) -> int:
Touches	Mostly local calculations or local UI state.
Calls detected	list, any, qn, get, iter, getparent, remove
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1674: def compact_generated_docx(doc: Document, keep_first_page_break: bool = True) -> int:
1675:     """Remove generator-inserted page breaks so pages fill naturally."""
1676:     kept = 0
1677:     removed = 0
1678:     for paragraph in list(doc.paragraphs):
1679:         has_page_break = any(
1680:             element.tag == qn("w:br") and element.get(qn("w:type")) == "page"
1681:             for element in paragraph._element.iter()
1682:         )
1683:         if not has_page_break:
1684:             continue
1685:         if keep_first_page_break and kept == 0:
1686:             kept += 1
1687:             continue
1688:         parent = paragraph._element.getparent()
1689:         if parent is not None:
1690:             parent.remove(paragraph._element)
1691:             removed += 1
1692:     return removed
1693:
1694:
```

write_single_code_reference_docx - lines 1695-1705

Item	Explanation
Source location	Lines 1695-1705 (11 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	def write_single_code_reference_docx(facts: dict, output_path: Path) -> None:
Touches	Mostly local calculations or local UI state.

Item	Explanation
Calls detected	setup_code_reference_document, add_source_fact_summary, add_source_fact_sections, compact_generated_docx, save
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1695: def write_single_code_reference_docx(facts: dict, output_path: Path) -> None:
1696:     doc = setup_code_reference_document(
1697:         f"Code Reference: {facts['repo_file']}",
1698:         "A source-level explanation with function details, file communication, variables, endpoints, and
debugging notes.",
1699:     )
1700:     add_source_fact_summary(doc, facts)
1701:     add_source_fact_sections(doc, facts, include_excerpts=True)
1702:     compact_generated_docx(doc, keep_first_page_break=False)
1703:     doc.save(output_path)
1704:
1705:
```

pdf_styles - lines 1706-1735

Item	Explanation
Source location	Lines 1706-1735 (30 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def pdf_styles():
Touches	Mostly local calculations or local UI state.
Calls detected	getSampleStyleSheet, add, ParagraphStyle, HexColor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1706: def pdf_styles():
1707:     styles = getSampleStyleSheet()
1708:     styles.add(ParagraphStyle(
1709:         name="CodeTiny",
1710:         parent=styles["Code"],
1711:         fontName="Courier",
1712:         fontSize=6.4,
1713:         leading=7.6,
1714:         backColor=colors.HexColor("#F5F8FB"),
1715:         borderColor=colors.HexColor("#D8E6F0"),
1716:         borderWidth=0.25,
1717:         borderPadding=4,
1718:         spaceAfter=7,
1719:     ))
1720:     styles.add(ParagraphStyle(
1721:         name="SmallBody",
1722:         parent=styles["BodyText"],
1723:         fontSize=8.5,
1724:         leading=11.0,
1725:         spaceAfter=5,
1726:     ))
1727:     styles.add(ParagraphStyle(
1728:         name="TinyTable",
1729:         parent=styles["BodyText"],
1730:         fontSize=7.0,
1731:         leading=8.5,
1732:     ))
1733:     return styles
1734:
1735:
```

pdf_paragraph - lines 1736-1739

Item	Explanation
Source location	Lines 1736-1739 (4 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def pdf_paragraph(text: str, style) -> Paragraph:
Touches	Mostly local calculations or local UI state.
Calls detected	Paragraph, escape, str, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1736: def pdf_paragraph(text: str, style) -> Paragraph:
1737:     return Paragraph(html.escape(str(text)).replace("\n", "<br/>"), style)
1738:
1739:
```

pdf_code - lines 1740-1748

Item	Explanation
Source location	Lines 1740-1748 (9 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def pdf_code(text: str, styles) -> Preformatted:
Touches	filesystem
Calls detected	in, splitlines, replace, wrap, extend, Preformatted, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1740: def pdf_code(text: str, styles) -> Preformatted:
1741:     wrapped_lines = []
1742:     for raw_line in (text or "").splitlines():
1743:         expanded = raw_line.replace("\t", "    ")
1744:         chunks = textwrap.wrap(expanded, width=112, replace_whitespace=False, drop_whitespace=False) or
[""]
1745:         wrapped_lines.extend(chunks)
1746:     return Preformatted("\n".join(wrapped_lines), styles["CodeTiny"])
1747:
1748:
```

pdf_table - lines 1749-1764

Item	Explanation
Source location	Lines 1749-1764 (16 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def pdf_table(rows: list[tuple[str, str]], styles):
Touches	Mostly local calculations or local UI state.
Calls detected	pdf_paragraph, extend, Table, setStyle, TableStyle, HexColor
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1749: def pdf_table(rows: list[tuple[str, str]], styles):
 1750:     data = [[pdf_paragraph("Item", styles["TinyTable"]), pdf_paragraph("Explanation",
styles["TinyTable"])]]
 1751:     data.extend([[pdf_paragraph(label, styles["TinyTable"]), pdf_paragraph(value, styles["TinyTable"])]
for label, value in rows])
 1752:     table = Table(data, colWidths=[1.45 * inch, 5.25 * inch], repeatRows=1)
 1753:     table.setStyle(TableStyle([
 1754:         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#E8EEF5")),
 1755:         ("GRID", (0, 0), (-1, -1), 0.25, colors.HexColor("#B7CFE0")),
 1756:         ("VALIGN", (0, 0), (-1, -1), "TOP"),
 1757:         ("LEFTPADDING", (0, 0), (-1, -1), 5),
 1758:         ("RIGHTPADDING", (0, 0), (-1, -1), 5),
 1759:         ("TOPPADDING", (0, 0), (-1, -1), 4),
 1760:         ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
 1761:     ]))
 1762:     return table
 1763:
 1764:
```

add_pdf_source_fact_sections - lines 1765-1803

Item	Explanation
Source location	Lines 1765-1803 (39 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_pdf_source_fact_sections(story: list, facts: dict, styles, include_excerpts: bool = True) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	append, Paragraph, pdf_paragraph, pdf_table, endpoint_purpose, function_detail_rows, pdf_code
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1765: def add_pdf_source_fact_sections(story: list, facts: dict, styles, include_excerpts: bool = True) ->
None:
 1766:     story.append(Paragraph("How this file participates in the system", styles["Heading2"]))
 1767:     story.append(pdf_paragraph("This generated section reads the actual file and points out line numbers,
 declarations, endpoints, events, source excerpts, and debugging cues.", styles["SmallBody"]))
 1768:     if facts["imports"]:
 1769:         story.append(Paragraph("Imports, requires, and external dependencies", styles["Heading2"]))
 1770:         story.append(pdf_table([(f"Line {item['line']}", item["statement"]) for item in
facts["imports"][:60]], styles))
 1771:     if facts["routes"]:
 1772:         story.append(Paragraph("Backend routes implemented here", styles["Heading2"]))
 1773:         story.append(pdf_table([(f"{item['method']} {item['path']}", f"Line {item['line']}.
{endpoint_purpose(item['path'])}") for item in facts["routes"][:80]], styles))
 1774:     if facts["endpoints"]:
 1775:         story.append(Paragraph("API endpoints mentioned or called", styles["Heading2"]))
 1776:         story.append(pdf_table([(item["endpoint"], f"Line {item['line']}.
{endpoint_purpose(item['endpoint'])}") for item in facts["endpoints"][:80]], styles))
 1777:     if facts["signals"]:
 1778:         story.append(Paragraph("Important implementation signals", styles["Heading2"]))
 1779:         story.append(pdf_table([(f"{item['type']} at line {item['line']}", f"{item['meaning']} Evidence:
{item['evidence']}") for item in facts["signals"][:80]], styles))
 1780:     if facts["variables"]:
 1781:         story.append(Paragraph("Important constants and variables", styles["Heading2"]))
 1782:         story.append(pdf_table([(item["name"], f"Line {item['line']}: {item['statement']}") for item in
facts["variables"][:80]], styles))
 1783:     if facts["json_keys"]:
 1784:         story.append(Paragraph("JSON/YAML-style keys detected", styles["Heading2"]))
 1785:         story.append(pdf_table([(item["key"], f"Line {item['line']}: {item['statement']}") for item in
facts["json_keys"][:80]], styles))
 1786:     if facts["events"]:
 1787:         story.append(Paragraph("Event handlers and user interactions", styles["Heading2"]))
```

```
 1788:         story.append(pdf_table([(f"Line {item['line']}", item["statement"]) for item in
facts["events"][:80]], styles))
 1789:     if facts["selectors"]:
 1790:         story.append(Paragraph("CSS selectors and visual hooks", styles["Heading2"]))
 1791:         story.append(pdf_table([(f"Line {item['line']}", item["selector"]) for item in
facts["selectors"][:120]], styles))
 1792:     if facts["functions"]:
 1793:         story.append(Paragraph("Function-by-function detail", styles["Heading2"]))
 1794:         for item in facts["functions"]:
 1795:             story.append(Paragraph(f"{item['name']} - lines {item['line']}-{item['end_line']}",
styles["Heading3"]))
 1796:             story.append(pdf_table(function_detail_rows(item), styles))
 1797:             if include_excerpts:
 1798:                 story.append(pdf_paragraph("Source excerpt:", styles["SmallBody"]))
 1799:                 story.append(pdf_code(item["excerpt"], styles))
 1800:     story.append(Paragraph("Representative opening snippet", styles["Heading2"]))
 1801:     story.append(pdf_code(facts["snippet"] or "(empty file)", styles))
 1802:
 1803:
```

write_single_code_reference_pdf - lines 1804-1827

Item	Explanation
Source location	Lines 1804-1827 (24 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	def write_single_code_reference_pdf(facts: dict, output_path: Path) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	pdf_styles, Paragraph, pdf_paragraph, pdf_table, fact_relationship_summary, str, len, Spacer, add_pdf_source_fact_sections, SimpleDocTemplate, build
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1804: def write_single_code_reference_pdf(facts: dict, output_path: Path) -> None:
 1805:     if not REPORTLAB_AVAILABLE:
 1806:         return
 1807:     styles = pdf_styles()
 1808:     story = [
 1809:         Paragraph(f"Code Reference: {facts['repo_file']}", styles["Title"]),
 1810:         pdf_paragraph("A source-level explanation with function details, file communication, variables,
endpoints, and debugging notes.", styles["SmallBody"]),
 1811:         pdf_table([
 1812:             ("File", facts["repo_file"]),
 1813:             ("Purpose", facts["description"]),
 1814:             ("Lines", f"{facts['line_count']:,}"),
 1815:             ("Size", f"{facts['size_bytes']:,} bytes"),
 1816:             ("Talks to", fact_relationship_summary(facts)),
 1817:             ("Functions", str(len(facts["functions"]))),
 1818:             ("Variables/constants", str(len(facts["variables"]))),
 1819:             ("API mentions", str(len(facts["endpoints"]))),
 1820:             ("Signals", str(len(facts["signals"]))),
 1821:         ], styles),
 1822:         Spacer(1, 0.12 * inch),
 1823:     ]
 1824:     add_pdf_source_fact_sections(story, facts, styles, include_excerpts=True)
 1825:     SimpleDocTemplate(str(output_path), pagesize=LETTER, rightMargin=0.55 * inch, leftMargin=0.55 * inch,
 topMargin=0.55 * inch, bottomMargin=0.55 * inch).build(story)
 1826:
 1827:
```

write_master_code_reference_docx - lines 1828-1856

Item	Explanation
Source location	Lines 1828-1856 (29 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	def write_master_code_reference_docx(code_files: list[str], facts_by_file: dict[str, dict], diagrams: dict[str, Path]) -> None:

Item	Explanation
Touches	network/API requests, authentication/security data
Calls detected	setup_code_reference_document, add_diagram, add_heading, numbers, add_table, len, signal, add_page_break, add_source_fact_summary, add_source_fact_sections, compact_generated_docx, save
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1828: def write_master_code_reference_docx(code_files: list[str], facts_by_file: dict[str, dict], diagrams:
dict[str, Path]) -> None:
 1829:     doc = setup_code_reference_document(
 1830:         "OMB Portfolio Builder Code Reference",
 1831:         "A generated Word reference that explains the important source files, all discovered functions,
communication paths, variables, endpoints, installers, AI paths, and compiler paths.",
 1832:     )
 1833:     add_diagram(doc, diagrams["file-communication-map"], "Main source file communication map.")
 1834:     add_diagram(doc, diagrams["frontend-backend-cloudflare"], "Frontend, backend, Cloudflare, and AI
boundary.")
 1835:     add_diagram(doc, diagrams["compile-code-flow"], "Compile Code workspace and simulator flow.")
 1836:     doc.add_heading("How to use this document", level=1)
 1837:     numbers(doc, [
 1838:         "Start with the file you want to understand.",
 1839:         "Read the summary table to understand its role.",
 1840:         "Read implementation signals to see whether the file touches Git, files, compilers, Cloudflare,
authentication, DOM, storage, or network calls.",
 1841:         "Read function details to understand line ranges, side effects, calls, endpoints, and source
excerpts.",
 1842:         "Use the debugging checklist to trace a behavior from click to function to file/API/output.",
 1843:     ])
 1844:     doc.add_heading("Generated file index", level=1)
 1845:     add_table(doc, [(repo_file, f"{facts_by_file[repo_file]['line_count']:,} lines,
{len(facts_by_file[repo_file]['functions'])} function(s), {len(facts_by_file[repo_file]['signals'])}
signal(s).") for repo_file in code_files])
 1846:     doc.add_page_break()
 1847:     for repo_file in code_files:
 1848:         facts = facts_by_file[repo_file]
 1849:         doc.add_heading(f"Source File: {repo_file}", level=1)
 1850:         add_source_fact_summary(doc, facts)
 1851:         add_source_fact_sections(doc, facts, include_excerpts=True)
 1852:         doc.add_page_break()
 1853:     compact_generated_docx(doc, keep_first_page_break=True)
 1854:     doc.save(CODE_REFERENCE_MASTER_DOCX)
 1855:
 1856:
```

write_master_code_reference_pdf - lines 1857-1895

Item	Explanation
Source location	Lines 1857-1895 (39 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	def write_master_code_reference_pdf(code_files: list[str], facts_by_file: dict[str, dict], diagrams: dict[str, Path]) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	pdf_styles, Paragraph, pdf_paragraph, get, exists, append, PdfImage, str, pdf_table, len, signal, PageBreak
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1857: def write_master_code_reference_pdf(code_files: list[str], facts_by_file: dict[str, dict], diagrams:
dict[str, Path]) -> None:
 1858:     if not REPORTLAB_AVAILABLE:
 1859:         return
 1860:     styles = pdf_styles()
```

```
 1861:     story = [
 1862:         Paragraph("OMB Portfolio Builder Code Reference", styles["Title"]),
 1863:         pdf_paragraph("A generated PDF reference that explains the important source files, discovered
functions, communication paths, variables, endpoints, installers, AI paths, and compiler paths.",
styles["SmallBody"]),
 1864:     ]
 1865:     for diagram_name, caption in [
 1866:         ("file-communication-map", "Main source file communication map."),
 1867:         ("frontend-backend-cloudflare", "Frontend, backend, Cloudflare, and AI boundary."),
 1868:         ("compile-code-flow", "Compile Code workspace and simulator flow."),
 1869:     ]:
 1870:         path = diagrams.get(diagram_name)
 1871:         if path and path.exists():
 1872:             story.append(PdfImage(str(path), width=6.6 * inch, height=3.72 * inch))
 1873:             story.append(pdf_paragraph(caption, styles["SmallBody"]))
 1874:     story.append(Paragraph("Generated file index", styles["Heading1"]))
 1875:     story.append(pdf_table([(repo_file, f"{facts_by_file[repo_file]['line_count']:,} lines,
{len(facts_by_file[repo_file]['functions'])} function(s), {len(facts_by_file[repo_file]['signals'])}
signal(s).") for repo_file in code_files], styles))
 1876:     story.append(PageBreak())
 1877:     for repo_file in code_files:
 1878:         facts = facts_by_file[repo_file]
 1879:         story.append(Paragraph(f"Source File: {repo_file}", styles["Heading1"]))
 1880:         story.append(pdf_table([
 1881:             ("File", facts["repo_file"]),
 1882:             ("Purpose", facts["description"]),
 1883:             ("Lines", f"{facts['line_count']:,}"),
 1884:             ("Size", f"{facts['size_bytes']:,} bytes"),
 1885:             ("Talks to", fact_relationship_summary(facts)),
 1886:             ("Functions", str(len(facts["functions"]))),
 1887:             ("Variables/constants", str(len(facts["variables"]))),
 1888:             ("API mentions", str(len(facts["endpoints"]))),
 1889:             ("Signals", str(len(facts["signals"]))),
 1890:         ], styles))
 1891:         add_pdf_source_fact_sections(story, facts, styles, include_excerpts=True)
 1892:         story.append(PageBreak())
 1893:     SimpleDocTemplate(str(CODE_REFERENCE_MASTER_PDF), pagesize=LETTER, rightMargin=0.55 * inch,
leftMargin=0.55 * inch, topMargin=0.55 * inch, bottomMargin=0.55 * inch).build(story)
 1894:
 1895:
```

write_code_reference_docs - lines 1896-1921

Item	Explanation
Source location	Lines 1896-1921 (26 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	def write_code_reference_docs(files: list[str], diagrams: dict[str, Path]) -> list[Path]:
Touches	filesystem
Calls detected	remove_directory, mkdir, is_text_code_file, extract_source_facts, code_reference_name, write_single_code_reference_docx, append, write_single_code_reference_pdf, exists, write_master_code_reference_docx, write_master_code_reference_pdf
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1896: def write_code_reference_docs(files: list[str], diagrams: dict[str, Path]) -> list[Path]:
 1897:     remove_directory(CODE_REFERENCE_DIR)
 1898:     remove_directory(CODE_REFERENCE_DOCX_DIR)
 1899:     remove_directory(CODE_REFERENCE_PDF_DIR)
 1900:     CODE_REFERENCE_DOCX_DIR.mkdir(parents=True, exist_ok=True)
 1901:     CODE_REFERENCE_PDF_DIR.mkdir(parents=True, exist_ok=True)
 1902:     code_files = [file for file in files if is_text_code_file(file)]
 1903:     facts_by_file = {repo_file: extract_source_facts(repo_file) for repo_file in code_files}
 1904:     written: list[Path] = []
 1905:     for repo_file in code_files:
 1906:         facts = facts_by_file[repo_file]
 1907:         docx_path = CODE_REFERENCE_DOCX_DIR / code_reference_name(repo_file, ".docx")
 1908:         write_single_code_reference_docx(facts, docx_path)
 1909:         written.append(docx_path)
 1910:         pdf_path = CODE_REFERENCE_PDF_DIR / code_reference_name(repo_file, ".pdf")
 1911:         write_single_code_reference_pdf(facts, pdf_path)
 1912:         if pdf_path.exists():
 1913:             written.append(pdf_path)
 1914:     write_master_code_reference_docx(code_files, facts_by_file, diagrams)
 1915:     written.append(CODE_REFERENCE_MASTER_DOCX)
```

```
 1916:     write_master_code_reference_pdf(code_files, facts_by_file, diagrams)
 1917:     if CODE_REFERENCE_MASTER_PDF.exists():
 1918:         written.append(CODE_REFERENCE_MASTER_PDF)
 1919:     return written
 1920:
 1921:
```

add_generated_code_reference - lines 1922-1971

Item	Explanation
Source location	Lines 1922-1971 (50 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_generated_code_reference(doc: Document, files: list[str], diagrams: dict[str, Path]) -> None:
Touches	authentication/security data
Calls detected	add_heading, write_code_reference_docs, add_paragraph, add_table, str, relative_to, exists, len, add_page_break, is_text_code_file, extract_source_facts, fact_relationship_summary
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1922: def add_generated_code_reference(doc: Document, files: list[str], diagrams: dict[str, Path]) -> None:
 1923:     doc.add_heading("Generated Word And PDF Code References", level=1)
 1924:     written = write_code_reference_docs(files, diagrams)
 1925:     doc.add_paragraph("The repository now includes generated Word and PDF code-reference documents rather
 than only Markdown notes. These documents are meant for source-level reading beside the actual files. They
explain functions, variables, API endpoints, imports, event handlers, file relationships,
security/authentication cues, compiler calls, installer behavior, and debugging paths.")
 1926:     add_table(doc, [
 1927:         ("Word folder", str(CODE_REFERENCE_DOCX_DIR.relative_to(REPO))),
 1928:         ("PDF folder", str(CODE_REFERENCE_PDF_DIR.relative_to(REPO))),
 1929:         ("Master Word reference", str(CODE_REFERENCE_MASTER_DOCX.relative_to(REPO))),
 1930:         ("Master PDF reference", str(CODE_REFERENCE_MASTER_PDF.relative_to(REPO)) if
CODE_REFERENCE_MASTER_PDF.exists() else "PDF generation skipped because ReportLab is unavailable."),
 1931:         ("Artifacts generated", str(len(written))),
 1932:         ("Regeneration command", "python docs/build_complete_guide.py"),
 1933:     ])
 1934:     doc.add_page_break()
 1935:
 1936:     for repo_file in [file for file in files if is_text_code_file(file)]:
 1937:         facts = extract_source_facts(repo_file)
 1938:         doc.add_heading(f"Code Reference: {repo_file}", level=1)
 1939:         doc.add_paragraph(facts["description"])
 1940:         add_table(doc, [
 1941:             ("Lines", f"{facts['line_count']:,}"),
 1942:             ("Size", f"{facts['size_bytes']:,} bytes"),
 1943:             ("Talks to", fact_relationship_summary(facts)),
 1944:             ("Functions discovered", str(len(facts["functions"]))),
 1945:             ("Variables discovered", str(len(facts["variables"]))),
 1946:             ("API endpoints mentioned", str(len(facts["endpoints"]))),
 1947:             ("Implementation signals", str(len(facts["signals"]))),
 1948:         ])
 1949:         if facts["signals"]:
 1950:             doc.add_heading("Signals detected", level=2)
 1951:             add_table(doc, [(f"{item['type']} at line {item['line']}", f"{item['meaning']} Evidence:
{item['evidence']}") for item in facts["signals"][:20]])
 1952:         if facts["endpoints"]:
 1953:             doc.add_heading("Endpoints in this file", level=2)
 1954:             add_table(doc, [(item["endpoint"], f"Line {item['line']}.
{endpoint_purpose(item['endpoint'])}") for item in facts["endpoints"][:20]])
 1955:         if facts["functions"]:
 1956:             doc.add_heading("Function details", level=2)
 1957:             for item in facts["functions"][:60]:
 1958:                 doc.add_heading(f"{item['name']} (lines {item['line']}-{item['end_line']})", level=3)
 1959:                 add_table(doc, function_detail_rows(item))
 1960:                 add_code(doc, item["excerpt"])
 1961:         if facts["variables"]:
 1962:             doc.add_heading("Variables and constants", level=2)
 1963:             add_table(doc, [(item["name"], f"Line {item['line']}: {item['statement']}") for item in
facts["variables"][:20]])
 1964:         if facts["imports"]:
 1965:             doc.add_heading("Imports and dependencies", level=2)
 1966:             add_table(doc, [(f"Line {item['line']}", item["statement"]) for item in
facts["imports"][:20]])
 1967:         doc.add_heading("Opening snippet", level=2)
 1968:         add_code(doc, facts["snippet"] or "(empty file)")
```

```
1969:         doc.add_page_break()
1970:
1971:
```

add_complete_function_inventory - lines 1972-1997

Item	Explanation
Source location	Lines 1972-1997 (26 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_complete_function_inventory(doc: Document) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	extract_functions, add_heading, add_paragraph, add_table, str, len, add_page_break, setdefault, append, items, range
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1972: def add_complete_function_inventory(doc: Document) -> None:
1973:     rows = extract_functions()
1974:     doc.add_heading("Generated Function Inventory", level=1)
1975:     doc.add_paragraph("This section is generated from the real source files. It does not replace reading
the code, but it gives you a map of where functions live and what their names suggest they own.")
1976:     add_table(doc, [
1977:         ("Files scanned", "main.cjs, server.mjs, template-preview.js, script.js"),
1978:         ("Functions found", str(len(rows))),
1979:         ("How descriptions are made", "Descriptions are heuristic based on file name and function naming
patterns. For exact behavior, inspect the source around the listed line."),
1980:     ])
1981:     doc.add_page_break()
1982:
1983:     by_file: dict[str, list[tuple[str, int, str, str]]] = {}
1984:     for row in rows:
1985:         by_file.setdefault(row[0], []).append(row)
1986:     for file_name, file_rows in by_file.items():
1987:         doc.add_heading(f"Function Inventory: {file_name}", level=1)
1988:         doc.add_paragraph(f"{file_name} contains {len(file_rows)} named functions discovered by the guide
generator.")
1989:         chunk_size = 45
1990:         for index in range(0, len(file_rows), chunk_size):
1991:             chunk = file_rows[index:index + chunk_size]
1992:             add_table(doc, [(f"{name} (line {line_no})", purpose) for _, line_no, name, purpose in
chunk])
1993:             if index + chunk_size < len(file_rows):
1994:                 doc.add_paragraph("Continued on the next table.")
1995:         doc.add_page_break()
1996:
1997:
```

add_deep_detail_topics - lines 1998-2011

Item	Explanation
Source location	Lines 1998-2011 (14 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_deep_detail_topics(doc: Document) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_heading, add_paragraph, bullets, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1998: def add_deep_detail_topics(doc: Document) -> None:
1999:     for title, body in DEEP_DETAIL_TOPICS:
2000:         doc.add_heading(f"Deep Detail: {title}", level=1)
2001:         doc.add_paragraph(body)
2002:         doc.add_heading("How to use this detail", level=2)
2003:         bullets(doc, [
2004:             "Start with the user-visible behavior.",
2005:             "Find the file that owns that behavior.",
2006:             "Trace the data handoff to the next file, endpoint, cache, or generated artifact.",
2007:             "Change the smallest responsible piece and test the flow end to end.",
2008:         ])
2009:         doc.add_page_break()
2010:
2011:
```

add_programming_syntax - lines 2012-2021

Item	Explanation
Source location	Lines 2012-2021 (10 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_programming_syntax(doc: Document) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_heading, add_code, add_paragraph, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2012: def add_programming_syntax(doc: Document) -> None:
2013:     for title, snippet, explanation in PROGRAMMING_SYNTAX_LESSONS:
2014:         doc.add_heading(f"Programming Syntax: {title}", level=1)
2015:         add_code(doc, snippet)
2016:         doc.add_paragraph(explanation)
2017:         doc.add_heading("How this appears in the project", level=2)
2018:         doc.add_paragraph("You will see this pattern in main.cjs, server.mjs, template-preview.js,
script.js, Cloudflare Worker code, HTML files, CSS files, JSON catalogs, and GitHub workflow YAML.")
2019:         doc.add_page_break()
2020:
2021:
```

add_shell_syntax - lines 2022-2031

Item	Explanation
Source location	Lines 2022-2031 (10 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_shell_syntax(doc: Document) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_heading, add_code, add_paragraph, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2022: def add_shell_syntax(doc: Document) -> None:
2023:     for title, snippet, explanation in SHELL_SYNTAX_LESSONS:
2024:         doc.add_heading(f"Shell Or Script Syntax: {title}", level=1)
2025:         add_code(doc, snippet)
2026:         doc.add_paragraph(explanation)
2027:         doc.add_heading("Why this matters", level=2)
2028:         doc.add_paragraph("Shell scripts glue tools together. They are not the app UI, but they install
```

```
tools, build releases, deploy workers, scan directories, and enforce rules.")
 2029:         doc.add_page_break()
 2030:
 2031:
```

add_caching_methods - lines 2032-2056

Item	Explanation
Source location	Lines 2032-2056 (25 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_caching_methods(doc: Document, diagrams: dict[str, Path]) -> None:
Touches	filesystem, authentication/security data
Calls detected	add_heading, add_diagram, add_paragraph, add_table, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2032: def add_caching_methods(doc: Document, diagrams: dict[str, Path]) -> None:
 2033:     doc.add_heading("Caching Methods Used In The Builder And Website", level=1)
 2034:     add_diagram(doc, diagrams["compile-code-flow"], "Compile artifact caching happens inside the Compile
Code workspace.")
 2035:     doc.add_paragraph("Caching means remembering a result so the app does not repeat slow or annoying
work. This project uses several different caches, each with a different safety boundary.")
 2036:     add_table(doc, [(name, f"{where} Purpose: {purpose}") for name, where, purpose in CACHING_METHODS])
 2037:     doc.add_heading("Important safety rule", level=2)
 2038:     doc.add_paragraph("A cache must be scoped. Publishing authorization is scoped to the same repository,
 branch, user/machine scope, and expiration time. Compiler caches are scoped to source code, language, compiler
path, runtime path, and file name. UI caches like update snooze are local convenience data and do not grant
publishing access.")
 2039:     doc.add_page_break()
 2040:     for name, where, purpose in CACHING_METHODS:
 2041:         doc.add_heading(f"Cache Detail: {name}", level=1)
 2042:         if "Compiler" in name or "Compile" in name or "Terminal" in name or "source" in name:
 2043:             add_diagram(doc, diagrams["compile-code-flow"], f"Context diagram for {name}.")
 2044:         elif "Publishing" in name or "Extended trust" in name:
 2045:             add_diagram(doc, diagrams["release-pipeline"], f"Publishing authorization cache supports this
 release/publish workflow.")
 2046:         elif "HTTP" in name:
 2047:             add_diagram(doc, diagrams["frontend-backend-cloudflare"], f"HTTP no-store protects request
freshness across frontend/backend paths.")
 2048:         add_table(doc, [
 2049:             ("Where implemented", where),
 2050:             ("Why it exists", purpose),
 2051:             ("What can go stale", "Cached data can become outdated when files, tools, versions,
repositories, or user choices change."),
 2052:             ("How to refresh", "Use the explicit refresh/rebuild/authenticate action for that feature,
clear the relevant local data, or restart the builder when appropriate."),
 2053:         ])
 2054:         doc.add_page_break()
 2055:
 2056:
```

add_installer_details - lines 2057-2092

Item	Explanation
Source location	Lines 2057-2092 (36 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_installer_details(doc: Document, diagrams: dict[str, Path]) -> None:
Touches	filesystem
Calls detected	add_heading, add_diagram, add_paragraph, add_table, numbers, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2057: def add_installer_details(doc: Document, diagrams: dict[str, Path]) -> None:
 2058:     doc.add_heading("How The Installer Works", level=1)
 2059:     add_diagram(doc, diagrams["release-pipeline"], "Installer and update behavior sits at the end of the
release pipeline.")
 2060:     doc.add_paragraph("The installer is produced by electron-builder and customized by NSIS script code
in build/installer.nsh. electron-builder gathers app files and creates the installer shell. NSIS controls
Windows setup pages, duplicate install checks, shortcut behavior, Git installation checks, update behavior, and
uninstall pages.")
 2061:     add_table(doc, [
 2062:         ("electron-builder", "Reads package.json build settings and packages the Electron app."),
 2063:         ("NSIS", "Runs installer wizard logic and custom script functions."),
 2064:         ("installer.nsh", "Adds OMB-specific setup pages, duplicate detection, update handling,
shortcuts, and Git checks."),
 2065:         ("AppData install", "Default per-user install location avoids Program Files admin prompts."),
 2066:         ("Update install", "Existing installation is updated in place instead of creating a duplicate
copy."),
 2067:     ])
 2068:     doc.add_heading("Installer flow", level=2)
 2069:     numbers(doc, [
 2070:         "customInit runs first.",
 2071:         "The installer checks for an existing registered install.",
 2072:         "If a current or newer install exists, setup stops.",
 2073:         "If an older install exists, setup treats the run as an update and uses the existing location.",
 2074:         "If no install exists, setup scans for duplicate builder copies.",
 2075:         "The custom tools page asks about desktop shortcut and publishing tools.",
 2076:         "Files are installed into the chosen per-user location.",
 2077:         "customInstall creates/removes shortcuts and optionally installs Git for Windows.",
 2078:         "The uninstaller has its own page explaining what will and will not be removed.",
 2079:     ])
 2080:     doc.add_page_break()
 2081:     for name, description in INSTALLER_FUNCTIONS:
 2082:         doc.add_heading(f"Installer Function: {name}", level=1)
 2083:         doc.add_paragraph(description)
 2084:         add_table(doc, [
 2085:             ("File", "build/installer.nsh"),
 2086:             ("Syntax family", "NSIS installer scripting"),
 2087:             ("Function job", description),
 2088:             ("Beginner note", "Installer functions run during setup/uninstall, not while the normal
builder UI is being used."),
 2089:         ])
 2090:         doc.add_page_break()
 2091:
 2092:
```

add_function_maps - lines 2093-2120

Item	Explanation
Source location	Lines 2093-2120 (28 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_function_maps(doc: Document, diagrams: dict[str, Path]) -> None:
Touches	filesystem
Calls detected	add_heading, add_diagram, add_paragraph, add_table, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2093: def add_function_maps(doc: Document, diagrams: dict[str, Path]) -> None:
 2094:     for file_name, group_purpose, functions in FUNCTION_GROUPS:
 2095:         doc.add_heading(f"Function Map: {file_name}", level=1)
 2096:         if file_name == "main.cjs":
 2097:             add_diagram(doc, diagrams["electron-runtime"], "Electron startup functions sit inside this
runtime flow.")
 2098:         elif file_name == "server.mjs":
 2099:             add_diagram(doc, diagrams["frontend-backend-cloudflare"], "server.mjs is the local backend
layer in this map.")
 2100:         elif file_name == "template-preview.js":
```

```
 2101:             add_diagram(doc, diagrams["builder-to-site-files"], "template-preview.js is the builder
frontend that sends edits toward parser output.")
 2102:         elif file_name == "script.js":
 2103:             add_diagram(doc, diagrams["cloudflare-ai-flow"], "script.js owns the public website behavior,
 including the AI chat request path.")
 2104:         doc.add_paragraph(group_purpose)
 2105:         add_table(doc, [(name, description) for name, description in functions])
 2106:         doc.add_heading("How to use this map", level=2)
 2107:         doc.add_paragraph("When debugging, find the user action first, then find the function group that
owns it. For example, a compile terminal issue starts in template-preview.js, then calls server.mjs, then
returns output back to template-preview.js.")
 2108:         doc.add_page_break()
 2109:         for name, description in functions:
 2110:             doc.add_heading(f"Function Detail: {file_name} -> {name}", level=1)
 2111:             doc.add_paragraph(description)
 2112:             add_table(doc, [
 2113:                 ("File", file_name),
 2114:                 ("Function", name),
 2115:                 ("Responsibility", description),
 2116:                 ("Debug question", "What input does this function receive, what output or state change
does it produce, and what function calls it next?"),
 2117:             ])
 2118:             doc.add_page_break()
 2119:
 2120:
```

add_architecture - lines 2121-2140

Item	Explanation
Source location	Lines 2121-2140 (20 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_architecture(doc: Document, diagrams: dict[str, Path]) -> None:
Touches	authentication/security data
Calls detected	add_heading, add_paragraph, get, add_diagram, add_code, bullets, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2121: def add_architecture(doc: Document, diagrams: dict[str, Path]) -> None:
 2122:     for title, body, diagram in ARCHITECTURE_PAGES:
 2123:         doc.add_heading(title, level=1)
 2124:         doc.add_paragraph(body)
 2125:         diagram_name = ARCHITECTURE_DIAGRAM_BY_TITLE.get(title)
 2126:         if diagram_name:
 2127:             add_diagram(doc, diagrams[diagram_name], f"Generated block diagram: {title}.")
 2128:         doc.add_heading("Text version of the block diagram", level=2)
 2129:         add_code(doc, diagram)
 2130:         doc.add_heading("Beginner explanation", level=2)
 2131:         doc.add_paragraph("Read each arrow as 'this thing talks to the next thing.' The important rule is
 that editing happens locally in the builder, while viewing happens publicly on the website after generated
static files are published.")
 2132:         doc.add_heading("What can break", level=2)
 2133:         bullets(doc, [
 2134:             "If local builder state is not saved, the parser may not see the latest edits.",
 2135:             "If Git authentication or branch state is wrong, Apply to site cannot safely push.",
 2136:             "If public hosting cache is stale, a phone may show an older copy until refresh or cache
expiry.",
 2137:         ])
 2138:         doc.add_page_break()
 2139:
 2140:
```

add_file_communication - lines 2141-2163

Item	Explanation
Source location	Lines 2141-2163 (23 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_file_communication(doc: Document, diagrams: dict[str, Path]) -> None:

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	add_heading, add_diagram, add_paragraph, add_table, bullets, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2141: def add_file_communication(doc: Document, diagrams: dict[str, Path]) -> None:
 2142:     doc.add_heading("How The Important Files Communicate", level=1)
 2143:     add_diagram(doc, diagrams["file-communication-map"], "Generated map of the main builder files and
public website files.")
 2144:     doc.add_paragraph("This is the file-level mental model. The desktop app starts in main.cjs, the
backend lives in server.mjs, the builder screen is template-preview.html with template-preview.css and template-
preview.js, and the public website is index.html with styles.css, script.js, projects.json, and assets.")
 2145:     add_table(doc, [
 2146:         ("main.cjs -> server.mjs", "Electron starts the backend so the builder UI can call local APIs."),
 2147:         ("main.cjs -> template-preview.html", "Electron opens the builder window and loads the builder
HTML file."),
 2148:         ("template-preview.html -> template-preview.js", "The builder HTML loads the JavaScript that
handles editing, saving, previewing, compile code, and publishing UI."),
 2149:         ("template-preview.js -> server.mjs", "The builder uses fetch requests to ask the backend to save
 files, parse projects, run Git, run compilers, or publish."),
 2150:         ("server.mjs -> projects.json", "The parser writes public project data used by the website."),
 2151:         ("index.html -> script.js/styles.css", "The public page loads its behavior and visual design."),
 2152:         ("script.js -> projects.json", "The public website reads the generated catalog to render project
windows."),
 2153:     ])
 2154:     doc.add_heading("Why this communication shape is useful", level=2)
 2155:     bullets(doc, [
 2156:         "The public website can stay static and fast.",
 2157:         "The builder can safely use local-only features without exposing them online.",
 2158:         "The parser creates a clean boundary between editable state and visitor-facing output.",
 2159:         "The same project data can be previewed locally before it is published.",
 2160:     ])
 2161:     doc.add_page_break()
 2162:
 2163:
```

add_tool_build_map - lines 2164-2173

Item	Explanation
Source location	Lines 2164-2173 (10 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_tool_build_map(doc: Document, diagrams: dict[str, Path]) -> None:
Touches	authentication/security data
Calls detected	add_heading, add_diagram, add_paragraph, add_table, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2164: def add_tool_build_map(doc: Document, diagrams: dict[str, Path]) -> None:
 2165:     doc.add_heading("What Tool Builds What", level=1)
 2166:     add_diagram(doc, diagrams["tool-build-map"], "Generated map showing which tool owns each part of the
build and release process.")
 2167:     doc.add_paragraph("A common source of confusion is thinking one tool builds everything. In this
project, each tool has a narrow job. pnpm installs dependencies, Electron runs the app, electron-builder
packages the app, NSIS makes the installer, Git moves files, GitHub Actions automates release builds, Wrangler
deploys Cloudflare Worker code, and compilers run project source code.")
 2168:     add_table(doc, [(tool, f"{builds} Used for: {where}") for tool, builds, where in TOOL_BUILD_ROWS])
 2169:     doc.add_heading("Beginner shortcut", level=2)
 2170:     doc.add_paragraph("When something fails, ask which tool owns that step. Installer failure points
toward NSIS or electron-builder. AI backend failure points toward Cloudflare Worker or Wrangler. Website publish
 failure points toward Git, GitHub, branch state, or authentication.")
```

```
2171:     doc.add_page_break()
2172:
2173:
```

add_software_decisions - lines 2174-2192

Item	Explanation
Source location	Lines 2174-2192 (19 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_software_decisions(doc: Document) -> None:
Touches	authentication/security data
Calls detected	add_heading, add_paragraph, add_table, bullets, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2174: def add_software_decisions(doc: Document) -> None:
2175:     for title, body in SOFTWARE_DECISIONS:
2176:         doc.add_heading(f"Software Engineering Decision: {title}", level=1)
2177:         doc.add_paragraph(body)
2178:         add_table(doc, [
2179:             ("Decision", title),
2180:             ("Reason", body),
2181:             ("What it protects", "Predictable publishing, safer public/private separation, easier
updates, clearer debugging, or better user experience."),
2182:             ("Tradeoff", "The design may add more files or moving parts, but each moving part has a
defined job."),
2183:         ])
2184:         doc.add_heading("How to evaluate this later", level=2)
2185:         bullets(doc, [
2186:             "If the feature becomes hard to maintain, check whether responsibilities are mixed across too
many files.",
2187:             "If users are confused, improve the builder UI or guide rather than exposing implementation
details.",
2188:             "If publishing becomes risky, tighten the parser and authentication boundary first.",
2189:         ])
2190:         doc.add_page_break()
2191:
2192:
```

add_system_layers - lines 2193-2208

Item	Explanation
Source location	Lines 2193-2208 (16 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_system_layers(doc: Document, diagrams: dict[str, Path]) -> None:
Touches	filesystem, authentication/security data
Calls detected	add_heading, add_diagram, add_paragraph, add_page_break, add_table, add_code
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2193: def add_system_layers(doc: Document, diagrams: dict[str, Path]) -> None:
2194:     doc.add_heading("AI, Frontend, Backend, Cloudflare, And Secrets", level=1)
2195:     add_diagram(doc, diagrams["frontend-backend-cloudflare"], "Generated map of the frontend, backend,
Cloudflare Worker, AI provider, and secret boundary.")
2196:     doc.add_paragraph("This section separates the major layers so you know where to look when something
breaks. The frontend draws and captures input. The backend performs protected work. Cloudflare hosts the public
```

```
AI endpoint. Secrets stay server-side.")
 2197:     doc.add_page_break()
 2198:     for title, body, rows, snippet in SYSTEM_LAYER_SECTIONS:
 2199:         doc.add_heading(title, level=1)
 2200:         doc.add_paragraph(body)
 2201:         add_table(doc, rows)
 2202:         doc.add_heading("Representative code or command", level=2)
 2203:         add_code(doc, snippet)
 2204:         doc.add_heading("Debug question", level=2)
 2205:         doc.add_paragraph("Ask which layer owns the problem: did the UI send the request, did the backend
 receive it, did the service have the needed secret or tool, and did the response come back in the format the UI
 expects?")
 2206:         doc.add_page_break()
 2207:
 2208:
```

add_workflows - lines 2209-2242

Item	Explanation
Source location	Lines 2209-2242 (34 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_workflows(doc: Document, files: list[str], diagrams: dict[str, Path]) -> None:
Touches	filesystem, authentication/security data
Calls detected	add_heading, add_diagram, numbers, add_paragraph, add_page_break, startswith, read_text, get, add_code, join, splitlines
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2209: def add_workflows(doc: Document, files: list[str], diagrams: dict[str, Path]) -> None:
 2210:     workflows = [
 2211:         ("Content Creation Workflow", ["Open the builder.", "Edit profile, front-page text, portfolio
areas, or projects.", "Save project or Save all sections.", "Open preview and inspect the exact public layout.",
 "Save draft.", "Apply to site after the target is authenticated."]),
 2212:         ("Project Creation Workflow", ["Click Add project.", "Choose category.", "Choose appearance.",
"Enter title.", "Edit overview and sections.", "Add files, images, links, code blocks, and compile-code
evidence.", "Save project so the parser can build the project preview."]),
 2213:         ("Publishing Workflow", ["Enter repository target.", "Authenticate GitHub write access.", "Load
from target if the target already has content.", "Save draft.", "Apply to site.", "Git commits and pushes
generated files.", "GitHub Pages or Cloudflare serves the updated website."]),
 2214:         ("Update Workflow", ["The installed app checks GitHub Releases.", "If a newer version exists, the
 app shows an update dialog.", "The updater downloads the installer.", "The app closes itself.", "The installer
updates the existing AppData install.", "The app relaunches after the installer finishes."]),
 2215:         ("Compile Code Workflow", ["Create or import a source file.", "Pick the language.", "For
Verilog/SystemVerilog, mark design files as Design and create or import a Testbench file.", "Keep $dumpfile and
$dumpvars in the HDL testbench so the signal scope can draw waveforms.", "Save source.", "Run beautifier if
desired.", "Compile or run.", "Read terminal output, messages log, and HDL scope when available.", "Append the
source code to a project section when ready."]),
 2216:     ]
 2217:     for title, steps in workflows:
 2218:         doc.add_heading(title, level=1)
 2219:         if title in {"Content Creation Workflow", "Project Creation Workflow", "Publishing Workflow"}:
 2220:             add_diagram(doc, diagrams["builder-to-site-files"], f"Generated flow diagram supporting
{title}.")
 2221:         if title == "Update Workflow":
 2222:             add_diagram(doc, diagrams["release-pipeline"], "Generated release/update pipeline diagram.")
 2223:         if title == "Compile Code Workflow":
 2224:             add_diagram(doc, diagrams["compile-code-flow"], "Generated Compile Code workflow diagram.")
 2225:         numbers(doc, steps)
 2226:         doc.add_heading("Plain-English mental model", level=2)
 2227:         doc.add_paragraph("A workflow is just a repeatable recipe. The builder turns complicated actions
into visible buttons so you do not need to remember shell commands every time.")
 2228:         doc.add_page_break()
 2229:
 2230:     for workflow in [f for f in files if f.startswith(".github/workflows/")]:
 2231:         text = (REPO / workflow).read_text(encoding="utf-8", errors="replace")
 2232:         doc.add_heading(f"GitHub Workflow File: {workflow}", level=1)
 2233:         doc.add_paragraph(FILE_DESCRIPTIONS.get(workflow, "GitHub Actions workflow file."))
 2234:         doc.add_heading("What a GitHub workflow is", level=2)
 2235:         doc.add_paragraph("A workflow is an automated script GitHub runs in the cloud. Instead of you
manually building installers or enforcing branch rules, GitHub reads this YAML file and executes each job on a
temporary machine.")
 2236:         doc.add_heading("Important YAML excerpt", level=2)
 2237:         add_code(doc, "\n".join(text.splitlines()[:80]))
 2238:         doc.add_heading("Tradeoff", level=2)
```

```
2239:         doc.add_paragraph("Automated workflows reduce mistakes, but a bad workflow can block good merges.
That is why the main branch gate should be strict but understandable.")
2240:         doc.add_page_break()
2241:
2242:
```

file_type - lines 2243-2250

Item	Explanation
Source location	Lines 2243-2250 (8 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def file_type(path: Path, repo_path: str) -> str:
Touches	Mostly local calculations or local UI state.
Calls detected	endswith, lower, lstrip
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2243: def file_type(path: Path, repo_path: str) -> str:
2244:     if repo_path.endswith(".gitkeep"):
2245:         return "folder marker"
2246:     if path.suffix:
2247:         return path.suffix.lower().lstrip(".")
2248:     return "no extension"
2249:
2250:
```

add_files - lines 2251-2280

Item	Explanation
Source location	Lines 2251-2280 (30 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_files(doc: Document, files: list[str]) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_heading, add_paragraph, get, add_table, file_type, stat, exists, is_file, lower, read_text, splitlines, add_code
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2251: def add_files(doc: Document, files: list[str]) -> None:
2252:     text_exts = {".js", ".mjs", ".cjs", ".html", ".css", ".json", ".md", ".txt", ".yml", ".yaml",
".toml", ".nsh", ""}
2253:     for repo_file in files:
2254:         path = REPO / repo_file
2255:         doc.add_heading(f"File Explained: {repo_file}", level=1)
2256:         doc.add_paragraph(FILE_DESCRIPTIONS.get(repo_file, "Tracked repository file used by the builder
or public website."))
2257:         add_table(doc, [
2258:             ("Type", file_type(path, repo_file)),
2259:             ("Size", f"{path.stat().st_size:,} bytes" if path.exists() else "missing locally"),
2260:             ("Used by", "Builder app, public site, installer, Cloudflare, or GitHub workflow depending on
the path."),
2261:             ("Beginner idea", "A repository is a folder Git tracks. This file is one object Git can
version, review, and publish."),
2262:         ])
2263:         if path.exists() and path.is_file() and path.suffix.lower() in text_exts:
2264:             lines = path.read_text(encoding="utf-8", errors="replace").splitlines()
2265:             doc.add_heading("Representative snippet", level=2)
```

```
 2266:             add_code(doc, "\n".join(lines[:24]) if lines else "(empty file)")
 2267:             if len(lines) > 24:
 2268:                 doc.add_paragraph(f"Only the first 24 lines are shown here. The full file has
{len(lines):,} lines and should be opened in the repository when editing.")
 2269:         else:
 2270:             doc.add_heading("Binary or asset note", level=2)
 2271:             doc.add_paragraph("This is treated as an asset. The guide explains its purpose, but the raw
binary content is not printed because raw image/icon bytes would not be useful to read in Word.")
 2272:         doc.add_heading("Safe editing rule", level=2)
 2273:         bullets(doc, [
 2274:             "Make changes on a feature branch from development.",
 2275:             "Preview or test the behavior before merging.",
 2276:             "Do not edit generated/public files by hand if the builder parser is supposed to generate
them.",
 2277:         ])
 2278:         doc.add_page_break()
 2279:
 2280:
```

add_commands - lines 2281-2295

Item	Explanation
Source location	Lines 2281-2295 (15 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_commands(doc: Document) -> None:
Touches	filesystem, authentication/security data
Calls detected	add_heading, add_code, add_table, add_paragraph, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2281: def add_commands(doc: Document) -> None:
 2282:     for command, explanation, why in COMMAND_LESSONS:
 2283:         doc.add_heading(f"Command Explained: {command}", level=1)
 2284:         add_code(doc, command)
 2285:         add_table(doc, [
 2286:             ("What it does", explanation),
 2287:             ("Why it matters", why),
 2288:             ("Where used", "Local development, GitHub Actions, builder publishing, installer testing, or
compiler execution."),
 2289:             ("Beginner warning", "Run commands from the correct folder. In this project, most development
 commands belong in the builder repository root."),
 2290:         ])
 2291:         doc.add_heading("How to read command output", level=2)
 2292:         doc.add_paragraph("Command output is the program talking back to you. Success output usually
names what happened. Error output usually names a missing file, missing tool, denied permission, wrong branch,
or failed compile step.")
 2293:         doc.add_page_break()
 2294:
 2295:
```

add_features - lines 2296-2318

Item	Explanation
Source location	Lines 2296-2318 (23 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_features(doc: Document, diagrams: dict[str, Path]) -> None:
Touches	filesystem, authentication/security data
Calls detected	add_heading, add_diagram, add_paragraph, lower, add_table, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2296: def add_features(doc: Document, diagrams: dict[str, Path]) -> None:
 2297:     for topic in FEATURE_TOPICS:
 2298:         doc.add_heading(f"Builder Feature: {topic}", level=1)
 2299:         if topic in {"Compile Code workspace", "HDL testbench requirement", "HDL signal scope", "Terminal
 output", "Messages log", "Append code to project"}:
 2300:             add_diagram(doc, diagrams["compile-code-flow"], f"Context diagram for {topic}.")
 2301:         elif topic in {"AI chat", "Cloudflare AI worker"}:
 2302:             add_diagram(doc, diagrams["cloudflare-ai-flow"], f"Context diagram for {topic}.")
 2303:         elif topic in {"Installer workflow", "Updater workflow", "Release tags", "Branch protection"}:
 2304:             add_diagram(doc, diagrams["release-pipeline"], f"Context diagram for {topic}.")
 2305:         elif topic in {"Project preview", "Full portfolio preview", "Project parser", "Asset
synchronization"}:
 2306:             add_diagram(doc, diagrams["builder-to-site-files"], f"Context diagram for {topic}.")
 2307:         doc.add_paragraph(f"This page explains the {topic.lower()} area of the OMB builder and website
system.")
 2308:         add_table(doc, [
 2309:             ("Owner view", "The builder exposes controls for creating, editing, saving, previewing, or
publishing this area."),
 2310:             ("Visitor view", "The public website shows only parsed, approved output. Editing controls
stay private."),
 2311:             ("Parser role", "The parser converts local builder state into website-safe data and
markup."),
 2312:             ("Risk", "If this area is not saved before preview or publishing, the public site may show an
 older version."),
 2313:         ])
 2314:         doc.add_heading("Plain-English example", level=2)
 2315:         doc.add_paragraph("Think of the builder as the kitchen and the public website as the plate served
 to visitors. The parser is the person plating the content neatly. Visitors should never see the prep
controls.")
 2316:         doc.add_page_break()
 2317:
 2318:
```

add_tradeoffs - lines 2319-2330

Item	Explanation
Source location	Lines 2319-2330 (12 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_tradeoffs(doc: Document) -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	add_heading, add_paragraph, add_table, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2319: def add_tradeoffs(doc: Document) -> None:
 2320:     for title, body in TRADEOFFS:
 2321:         doc.add_heading(f"Tradeoff: {title}", level=1)
 2322:         doc.add_paragraph(body)
 2323:         add_table(doc, [
 2324:             ("Benefit", "The chosen direction fits the current builder goal: owner-controlled, local-
first, preview-before-publish portfolio creation."),
 2325:             ("Cost", "Every architecture choice gives up something. Usually the cost is complexity, setup
 work, or less flexibility in another direction."),
 2326:             ("Decision rule", "Prefer the option that keeps publishing safe, previews accurate, and
editing understandable."),
 2327:         ])
 2328:         doc.add_page_break()
 2329:
 2330:
```

add_troubleshooting - lines 2331-2343

Item	Explanation
Source location	Lines 2331-2343 (13 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_troubleshooting(doc: Document) -> None:
Touches	filesystem
Calls detected	add_heading, add_paragraph, numbers, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2331: def add_troubleshooting(doc: Document) -> None:
2332:     for title, body in TROUBLESHOOTING_TOPICS:
2333:         doc.add_heading(f"Troubleshooting: {title}", level=1)
2334:         doc.add_paragraph(body)
2335:         numbers(doc, [
2336:             "Read the exact error message before changing anything.",
2337:             "Confirm whether the issue is local builder state, Git state, installer state, public website
cache, or backend service availability.",
2338:             "Fix the smallest proven cause first.",
2339:             "Save a clean checkpoint after the fix works.",
2340:         ])
2341:         doc.add_page_break()
2342:
2343:
```

add_concepts - lines 2344-2357

Item	Explanation
Source location	Lines 2344-2357 (14 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_concepts(doc: Document) -> None:
Touches	filesystem
Calls detected	add_heading, add_paragraph, add_table, add_page_break
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 2344: def add_concepts(doc: Document) -> None:
 2345:     for concept in CONCEPTS:
 2346:         doc.add_heading(f"Beginner Concept: {concept}", level=1)
 2347:         doc.add_paragraph(f"{concept} is one of the building blocks behind the OMB builder and portfolio
website. You do not need to memorize every term at once. Learn the role it plays, then come back when you see it
 in an error message or file name.")
 2348:         add_table(doc, [
 2349:             ("Simple meaning", "This concept helps explain how the app, website, GitHub workflow,
installer, or backend behaves."),
 2350:             ("Where it appears", "Look for it in README instructions, source files, GitHub Actions logs,
installer behavior, browser behavior, or terminal output."),
 2351:             ("How to debug it", "Ask: what is supposed to happen, what actually happened, what file or
service owns that behavior, and what changed most recently?"),
 2352:         ])
 2353:         doc.add_heading("Tiny mental model", level=2)
 2354:         doc.add_paragraph("When confused, draw boxes. Put the user on the left, the thing they click in
the middle, and the file or service that responds on the right. Most software problems become easier once the
boxes are visible.")
 2355:         doc.add_page_break()
 2356:
 2357:
```

add_closing - lines 2358-2372

Item	Explanation
Source location	Lines 2358-2372 (15 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def add_closing(doc: Document) -> None:
Touches	filesystem
Calls detected	add_heading, numbers, add_note
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2358: def add_closing(doc: Document) -> None:
2359:     doc.add_heading("Final Operating Checklist", level=1)
2360:     numbers(doc, [
2361:         "Make feature changes on a feature or codex branch from development.",
2362:         "Update README and the in-app guide when behavior changes.",
2363:         "Run local syntax checks and useful smoke tests.",
2364:         "Merge feature work into development.",
2365:         "Only merge development into main for release-ready builds.",
2366:         "Bump package.json before a new main release.",
2367:         "Confirm GitHub Actions publish a fresh installer and stable latest asset.",
2368:         "Open the builder, check update behavior, and verify the public website link downloads the latest
installer.",
2369:     ])
2370:     add_note(doc, "The core rule", "The builder is the source of truth for creating content. The preview
is the truth test before publishing. GitHub is the delivery path. The public website is the final read-only
result.")
2371:
2372:
```

main - lines 2373-2410

Item	Explanation
Source location	Lines 2373-2410 (38 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	def main() -> None:
Touches	Mostly local calculations or local UI state.
Calls detected	tracked_files, load_package, make_diagrams, setup_document, add_cover, add_reading_map, add_entry_points, add_architecture, add_flow_walkthroughs, add_file_communication, add_tool_build_map, add_software_decisions
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2373: def main() -> None:
2374:     files = tracked_files()
2375:     package = load_package()
2376:     diagrams = make_diagrams()
2377:     doc = setup_document()
2378:     add_cover(doc, package)
2379:     add_reading_map(doc)
2380:     add_entry_points(doc)
2381:     add_architecture(doc, diagrams)
2382:     add_flow_walkthroughs(doc, diagrams)
2383:     add_file_communication(doc, diagrams)
2384:     add_tool_build_map(doc, diagrams)
2385:     add_software_decisions(doc)
2386:     add_system_layers(doc, diagrams)
```

```
 2387:     add_data_contracts(doc)
 2388:     add_api_endpoint_catalog(doc)
 2389:     add_programming_syntax(doc)
 2390:     add_shell_syntax(doc)
 2391:     add_caching_methods(doc, diagrams)
 2392:     add_installer_details(doc, diagrams)
 2393:     add_function_maps(doc, diagrams)
 2394:     add_complete_function_inventory(doc)
 2395:     add_generated_code_reference(doc, files, diagrams)
 2396:     add_deep_detail_topics(doc)
 2397:     add_workflows(doc, files, diagrams)
 2398:     add_files(doc, files)
 2399:     add_commands(doc)
 2400:     add_features(doc, diagrams)
 2401:     add_tradeoffs(doc)
 2402:     add_troubleshooting(doc)
 2403:     add_concepts(doc)
 2404:     add_closing(doc)
 2405:     removed_breaks = compact_generated_docx(doc, keep_first_page_break=True)
 2406:     print(f"Compacted generated DOCX layout by removing {removed_breaks} page breaks.")
 2407:     doc.save(OUTPUT)
 2408:     print(OUTPUT)
 2409:
 2410:
```

Representative opening snippet

```
    1: from __future__ import annotations
    2:
    3: import html
    4: import json
    5: import math
    6: import re
    7: import shutil
    8: import subprocess
    9: import textwrap
   10: from pathlib import Path
   11:
   12: from PIL import Image, ImageDraw, ImageFont
   13: from docx import Document
   14: from docx.enum.table import WD_CELL_VERTICAL_ALIGNMENT, WD_TABLE_ALIGNMENT
   15: from docx.enum.text import WD_ALIGN_PARAGRAPH
   16: from docx.oxml import OxmlElement
   17: from docx.oxml.ns import qn
   18: from docx.shared import Inches, Pt, RGBColor
   19:
   20: try:
   21:     from reportlab.lib import colors
   22:     from reportlab.lib.pagesizes import LETTER
   23:     from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
   24:     from reportlab.lib.units import inch
   25:     from reportlab.platypus import Image as PdfImage
   26:     from reportlab.platypus import PageBreak, Paragraph, Preformatted, SimpleDocTemplate, Spacer, Table,
TableStyle
   27:
   28:     REPORTLAB_AVAILABLE = True
   29: except ImportError:
   30:     REPORTLAB_AVAILABLE = False
   31:
   32:
   33: REPO = Path(__file__).resolve().parents[1]
   34: OUTPUT = REPO / "docs" / "OMB_Portfolio_Builder_Complete_Guide.docx"
   35: DIAGRAM_DIR = REPO / "docs" / "guide-diagrams"
   36: CODE_REFERENCE_DIR = REPO / "docs" / "code-reference"
   37: CODE_REFERENCE_DOCX_DIR = REPO / "docs" / "code-reference-docx"
   38: CODE_REFERENCE_PDF_DIR = REPO / "docs" / "code-reference-pdf"
   39: CODE_REFERENCE_MASTER_DOCX = REPO / "docs" / "OMB_Portfolio_Builder_Code_Reference.docx"
   40: CODE_REFERENCE_MASTER_PDF = REPO / "docs" / "OMB_Portfolio_Builder_Code_Reference.pdf"
   41:
   42:
   43: FILE_DESCRIPTIONS = {
   44:     ".github/workflows/build-windows-builder.yml": "GitHub Actions workflow that builds the Windows
installer and portable application, checks the stable download link, enforces release version bumps, uploads
artifacts, and publishes GitHub Releases.",
   45:     ".github/workflows/main-branch-gate.yml": "GitHub Actions guard that rejects direct pushes to main
and rejects pull requests into main unless the source branch is development.",
   46:     ".gitignore": "Git ignore rules for build outputs, temporary files, and local-only data that should
not be committed.",
   47:     ".nojekyll": "Tells GitHub Pages not to process the site with Jekyll, so folders and static files
publish exactly as generated.",
   48:     ".well-known/security.txt": "Public security contact file used by browsers, researchers, and scanners
 to find responsible disclosure information.",
   49:     "BRANCHING.md": "Human-readable branch model explaining development, main, feature branches, and
release flow.",
   50:     "Backgrounds/.gitkeep": "Keeps the Backgrounds folder in Git even when no background image files are
present.",
   51:     "README.md": "Main README for the standalone builder application: install, updates, workspaces, text
editing, publishing, build commands, branch model, and uninstall.",
   52:     "_headers": "Cloudflare Pages style HTTP headers for security and caching behavior on the public
website.",
```

```
   53:     "assets/apple-touch-icon.png": "Apple touch icon shown when the public site is saved on iOS or
compatible launch surfaces.",
   54:     "assets/favicon-16.png": "Small browser favicon used by tabs and browser UI.",
   55:     "assets/favicon-192.png": "Large web app icon for Android and installable browser surfaces.",
   56:     "assets/favicon-32.png": "Standard browser favicon size.",
   57:     "assets/favicon-48.png": "Windows/browser icon size used by some desktop surfaces.",
   58:     "assets/favicon.ico": "ICO favicon bundle used by browsers that request favicon.ico.",
   59:     "assets/omb-app-icon-256.png": "Builder application icon source at 256 pixels.",
   60:     "assets/omb-app-icon-512.png": "Builder application icon source at 512 pixels.",
   61:     "assets/omb-app-icon.ico": "Windows ICO application icon used by Electron Builder and shortcuts.",
   62:     "assets/omb-mark.png": "OMB snake/mark image used for branding in the site and builder.",
   63:     "build/installer.nsh": "NSIS installer customization script for update behavior, shortcut placement,
old install detection, and setup details.",
   64:     "builder-rich-future-sections.js": "Builder-side helper code for richer future portfolio sections and
 section defaults.",
   65:     "cloudflare/README.md": "Cloudflare setup notes for deploying the AI Worker endpoint and wiring the
public site to the backend.",
   66:     "cloudflare/portfolio-ai-worker.js": "Cloudflare Worker backend for the public portfolio AI assistant
 and fallback intelligence path.",
   67:     "cloudflare/wrangler.toml": "Wrangler configuration used to deploy the Cloudflare Worker.",
   68:     "docs/.gitkeep": "Keeps the docs folder in Git even before generated documents are added.",
   69:     "electronics-search.js": "Electronics vocabulary and search support used by the website search
behavior.",
   70:     "index.html": "Public website HTML shell that recruiters and visitors load in the browser.",
   71:     "installer-notes.txt": "Installer license/notes text shown during Windows setup.",
   72:     "main.cjs": "Electron main process: creates the desktop window, starts the local backend, handles
menus, update handoff, and application lifecycle.",
```